

REDVITAL

Plataforma Nacional para la Gestión Integral del Ciclo de Vida
de la Donación de Sangre

HERRAMIENTAS, POLÍTICAS Y LINEAMIENTOS

Arkhé Software S.A.S.

De la raíz a la red

Curso: Arquitectura de Software Empresarial
Cliente: Profesor de la asignatura, en el rol simulado de
Ministerio de Salud y Protección Social
Entrega: Review Sprint 2 y Planning Sprint 3 — Semana 8
Versión: 3.0
Estado: Aprobado para entrega
Fecha: 22 de septiembre de 2026
Clasificación: Uso interno del proyecto

Control de versiones

Versión	Fecha	Autor	Descripción del cambio
1.0	16/08/2026	Equipo Arkhé	Versión inicial. Catálogo de herramientas, plantillas de documentación y políticas por fase y de DevOps.
2.0	29/08/2026	Equipo Arkhé	Alineación con las restricciones de diseño RD-01 a RD-08. Arquitectura distribuida de servicios de grano grueso. Incorporación del stack .NET y del API Gateway. Adopción de GitFlow y de organización multi-repositorio. Nuevas secciones de contenerización, observabilidad y gestión del backlog.
2.1	29/08/2026	Equipo Arkhé	Las restricciones de diseño pasan a sección propia con regla de precedencia. Se incorporan glosario, referencias, plan de adopción, definición de listo y gobierno del documento.
2.2	30/08/2026	Equipo Arkhé	Auditoría de vigencia del instrumental. Se unifica el criterio de versión entre las dos plataformas de servidor y se ajustan las líneas adoptadas a las que tienen soporte de seguridad vigente. Se incorporan las cadenas de construcción de ambas plataformas y el destino de despliegue de la demostración. Se añaden al Tech Radar las reglas de ubicación y de vigencia. El catálogo queda en 73 entradas.

Continúa en la página siguiente

Versión	Fecha	Autor	Descripción del cambio
3.0	22/09/2026	Equipo Arkhé	Incorporación de ADR-007 a ADR-016. El catálogo refleja la descomposición en cinco servicios de ADR-014: doce contenedores en Desarrollo, catorce en QA, cuatro trabajos de migración y nueve repositorios. Entran Caddy como proxy de borde (74) y nginx en anillo Evaluar (75), por ADR-015; Grype para el escaneo de imágenes (76) y Gitleaks para la detección de secretos (77); la bandeja de salida transaccional expuesta por contrato (78), por ADR-014; y Trivy en anillo Evitar (79). Por alineación con el Documento de Infraestructura V1 entran Gatling para las pruebas de carga (80) y las imágenes de ejecución mínimas <i>chiseled</i> y <i>distroless</i> (81), y se actualizan las entradas 1, 26, 27, 30, 31, 32 y 74. Las entradas 1, 2, 11 y 29 incorporan lo registrado en ADR-009, ADR-007 y ADR-008. Se fija la regla de numeración estable. El catálogo queda en 81 entradas. Los ambientes se unifican en Desarrollo, QA —que es la máquina de demostración— y Producción declarada; la prueba de carga se ejecuta sobre una pila de medición separada. Containerización, integración continua, observabilidad y seguridad operativa incorporan el endurecimiento de contenedores, las redes por zona, la fijación de acciones de terceros y la rotación de la clave de firma. La definición de listo incorpora el criterio de ausencia de contraprestación económica para las historias de reconocimiento al donante, y los indicadores de flujo incorporan la equivalencia entre el punto de historia y las horas de capacidad.

Aprobaciones

Rol	Responsable	Firma / Fecha
Product Owner	Alexander Aponte	
Scrum Master / Project Manager	Carlos Santiago Pinzón	
Arquitecta de Software	Sara	
QA / Tester	Tomás	
DevOps	Veli	
Desarrollador	Sebastián	

DesarrolladorDamián

La aprobación de la Arquitecta de Software y del DevOps es obligatoria: son los dos roles cuyo trabajo queda normado de forma directa por este documento. La del Product Owner acredita que las restricciones del cliente quedaron transcritas sin alteración.

Índice

Control de versiones	1
Aprobaciones	2
1. Introducción	7
1.1. Propósito	7
1.2. Alcance y relación con otros documentos	7
1.3. Audiencia y uso	8
1.4. Definiciones, acrónimos y abreviaturas	8
1.5. Referencias	10
2. Restricciones de diseño	11
2.1. Restricciones del cliente	11
2.2. Restricciones normativas y del dominio	12
2.3. Condiciones del proyecto	13
2.4. Regla de precedencia entre restricciones	14
3. Tech Radar	14
3.1. Metodología	14
3.2. Radar visual	16
3.3. Cuadrante 1 — Lenguajes y Frameworks	17
3.4. Cuadrante 2 — Datos e Infraestructura	25
3.5. Cuadrante 3 — Gestión y Colaboración	36
3.6. Cuadrante 4 — Técnicas y Prácticas	39
3.7. Resumen del catálogo	44
4. Plan de adopción	44
4.1. Cronograma del proyecto	44
4.2. Adopción por sprint	45
4.3. Riesgos de adopción	47
5. Plantillas para la documentación	49
5.1. Plantilla de Registro de Decisión Arquitectónica	49
5.2. Plantilla de Historia de Usuario	50
5.3. Plantilla de registro de riesgos	51
5.4. Estructura del Documento de Diseño de Software	51
5.5. Otras plantillas vigentes	52
6. Políticas y lineamientos por fase del proyecto	52
6.1. Fase 1 — Iniciación y formulación	52
6.2. Fase 2 — Diseño y arquitectura	53
6.3. Fase 3 — Construcción	53
6.4. Fase 4 — Pruebas	54
6.5. Fase 5 — Estabilización y entrega	54
7. Políticas y lineamientos de DevOps	54
7.1. Organización de repositorios	54
7.2. Control de versiones y estrategia de ramificación	55
7.3. Revisión de código	57

7.4. Contenerización	57
7.5. Integración y despliegue continuo	59
7.6. Observabilidad	61
7.7. Seguridad operativa	61
7.8. Trazabilidad entre artefactos	62
8. Gestión del backlog y métricas	63
8.1. Herramienta y estructura	63
8.2. Definición de listo y definición de terminado	63
8.3. Métricas de flujo	64
9. Gobierno del documento	65
A. Matriz de cumplimiento de restricciones	66

Índice de figuras

1.	Tech Radar de RedVital, versión 3.0. Cada punto es una entrada del catálogo; la numeración corresponde a las tablas de las secciones 3.3 a 3.6. Cuanto más cerca del centro, mayor es el compromiso del equipo con esa entrada.	16
2.	Flujo de ramas adoptado. Las funcionalidades nacen de develop , las publicaciones se estabilizan en release y se etiquetan en main , y las correcciones urgentes retornan a las dos ramas de larga vida.	56
3.	Cadena de trazabilidad entre artefactos del proyecto. La etiqueta de la imagen desplegada permite recorrer la cadena en sentido inverso hasta el requerimiento de origen.	63

Índice de cuadros

3.	Relación con los demás artefactos del proyecto.	7
4.	Glosario de términos y acrónimos empleados en este documento.	8
5.	Normas, estándares y fuentes que este documento aplica.	10
6.	Restricciones de diseño impuestas por el cliente (RD-01 a RD-08).	12
7.	Restricciones normativas y del dominio (RN-01 a RN-04).	13
8.	Condiciones del proyecto que acotan la selección (CP-01 a CP-04).	13
9.	Significado de los cuatro anillos.	15
10.	Catálogo del cuadrante Lenguajes y Frameworks (entradas 1 a 25 y 80).	17
11.	Catálogo del cuadrante Datos e Infraestructura (entradas 26 a 44, 74, 75 y 81).	26
12.	Catálogo del cuadrante Gestión y Colaboración (entradas 45 a 53, 76, 77 y 79).	36
13.	Catálogo del cuadrante Técnicas y Prácticas (entradas 54 a 73 y 78).	40
14.	Distribución de las 81 entradas por cuadrante y anillo.	44
15.	Cronograma de entregas y ubicación de los sprints.	45
16.	Plan de incorporación del catálogo, sprint por sprint.	45
17.	Riesgos derivados del instrumental y de las políticas.	47
18.	Campos obligatorios de un ADR.	49
19.	Campos obligatorios de una historia del backlog.	50
20.	Campos del registro de riesgos del proyecto.	51
21.	Plantillas complementarias y su uso.	52
22.	Repositorios de la organización y su responsabilidad.	55
23.	Reglas por rama.	56
24.	Convenciones de commit, versionado y frecuencia.	56
25.	Lineamientos de contenerización.	58
26.	Lineamientos de integración y despliegue continuo.	60
27.	Estructura y reglas del backlog.	63
28.	Definición de listo (DoR) y definición de terminado (DoD).	64
29.	Indicadores de flujo del proyecto y su fuente.	64
30.	Reglas de gobierno de este documento.	65
31.	Matriz de cumplimiento de las restricciones de diseño y normativas.	66

1. Introducción

1.1 Propósito

Este documento define el conjunto de herramientas, plantillas, políticas y lineamientos que rigen el desarrollo del proyecto RedVital. Su propósito es que cualquier integrante del equipo, o alguien que se incorpore después, sepa con qué se trabaja, por qué se eligió cada cosa, y bajo qué reglas se produce y se entrega el software.

El documento se organiza en seis partes:

1. **Restricciones de diseño** (Sección 2): las restricciones del cliente, las normativas y las condiciones del proyecto que acotan toda elección posterior.
2. **Tech Radar** (Sección 3): catálogo de herramientas y prácticas con anillo, uso y justificación.
3. **Plan de adopción** (Sección 4): en qué sprint entra cada elemento del catálogo y qué riesgos trae.
4. **Plantillas de documentación** (Sección 5): formatos estándar de los artefactos que produce el equipo.
5. **Políticas por fase y de DevOps** (Secciones 6 y 7): reglas de trabajo, control de versiones, revisión, contenerización, integración continua, observabilidad y seguridad operativa.
6. **Gestión del backlog y gobierno** (Secciones 8 y 9): cómo se administra el trabajo, de dónde salen los indicadores y cómo cambia este documento.

1.2 Alcance y relación con otros documentos

Este artefacto no especifica *qué* construye el sistema, eso corresponde al SRS, ni *cómo* se estructura internamente, eso corresponde al SAD y al SDD. Define únicamente el instrumental de trabajo y las reglas de producción.

Cuando una decisión de este documento tiene consecuencias estructurales, se remite al Registro de Decisión Arquitectónica correspondiente, que es el artefacto donde esa decisión se delibera y se sustenta. La relación es de una sola dirección: el ADR decide, este documento registra la consecuencia sobre el instrumental. Un cambio de anillo nunca se acuerda aquí; se acuerda en la Mesa de Arquitectura y aquí se anota.

Cuadro 3: Relación con los demás artefactos del proyecto.

Artefacto	Qué define, y qué toma de este documento
Documento de Proyecto	Alcance, cronograma, equipo, presupuesto y riesgos de gestión. Toma de aquí el rubro de licencias, que este documento reduce a cero, y el de infraestructura, que este documento reduce a la máquina virtual institucional.
SRS	Requerimientos funcionales y no funcionales, usuarios y matriz de acceso. Comparte con este documento las restricciones de la Sección 2, que deben aparecer idénticas en ambos.
SAD / SDD	Arquitectura, vistas 4+1, modelo C4, modelo de datos y escenarios de calidad. Toma de aquí las tecnologías en anillo Adoptar; no puede usar ninguna que no esté en el catálogo.

Continúa en la página siguiente

Tabla 3 — continuación

Artefacto	Qué define, y qué toma de este documento
ADR	Deliberación y sustento de cada decisión estructural. Es la fuente; este documento es la consecuencia.
Backlog	Trabajo comprometido. Toma de aquí la plantilla de historia, la definición de listo y la definición de terminado.

1.3 Audiencia y uso

Está dirigido a los siete integrantes del equipo y al cliente. Se consulta en tres momentos concretos: antes de incorporar una dependencia o una herramienta, antes de abrir una rama, y antes de dar por terminada una historia. Fuera de esos tres momentos no hace falta leerlo entero.

1.4 Definiciones, acrónimos y abreviaturas

Cuadro 4: Glosario de términos y acrónimos empleados en este documento.

Término	Definición
ACME	<i>Automatic Certificate Management Environment</i> . Protocolo con el que el proxy de borde obtiene y renueva certificados de una autoridad pública sin intervención manual.
ADR	<i>Architecture Decision Record</i> . Registro estructurado de una decisión arquitectónica, su contexto, sus alternativas y sus consecuencias.
Anillo	Nivel de compromiso del equipo con una entrada del Tech Radar: Adoptar, Probar, Evaluar o Evitar.
Bandeja de salida	Tabla de eventos notificables que el servicio emisor escribe en la misma transacción que el hecho que los origina, y que el trabajador de Notificaciones consume por contrato.
Contexto acotado	<i>Bounded context</i> . Frontera dentro de la cual un modelo de dominio es coherente y un término tiene un único significado. Define dónde se corta un servicio.
CP	Condición del Proyecto. Restricción originada en el equipo, el tiempo o el presupuesto, no en el cliente.
Cuadrante	Categoría a la que pertenece una entrada del Tech Radar: lenguajes, datos, gestión o prácticas.
DoD	<i>Definition of Done</i> . Condiciones que debe cumplir una historia para considerarse terminada.
DoR	<i>Definition of Ready</i> . Condiciones que debe cumplir una historia para poder entrar a un sprint.
EOL	<i>End of Life</i> . Fecha a partir de la cual una versión deja de recibir parches de seguridad gratuitos.

Continúa en la página siguiente

Tabla 4 — continuación

Término	Definición
GHCR	<i>GitHub Container Registry</i> . Registro de imágenes de contenedor de la organización.
GitFlow	Estrategia de ramificación con ramas de larga vida, de publicación y de corrección urgente.
HU	Historia de usuario. Unidad de trabajo del backlog.
INVEST	Criterios de calidad de una historia: independiente, negociable, valiosa, estimable, pequeña y verificable.
JWKS	<i>JSON Web Key Set</i> . Conjunto de claves públicas que publica el emisor de un token para que otros lo validen sin consultarlo.
JWT	<i>JSON Web Token</i> . Token firmado que transporta la identidad, el rol y la jurisdicción del usuario.
Jurisdicción	Ámbito territorial sobre el que un usuario tiene visibilidad de datos.
LTS	<i>Long Term Support</i> . Versión de una plataforma con ventana de soporte extendida.
MoSCoW	Priorización del alcance en <i>Must, Should, Could</i> y <i>Won't</i> .
OpenAPI	Especificación estándar para describir una interfaz REST de forma legible por máquina.
ORM	<i>Object-Relational Mapping</i> . Capa que traduce entre objetos del lenguaje y tablas relacionales.
PR	<i>Pull Request</i> . Solicitud de fusión de una rama, sujeta a revisión y a verificación automática.
RD	Restricción de Diseño. Restricción impuesta por el cliente sobre cómo debe construirse la solución.
RF / RNF	Requerimiento Funcional / Requerimiento No Funcional, definidos en el SRS.
RN	Restricción Normativa. Restricción originada en el marco legal colombiano o en una norma técnica. No confundir con las reglas de negocio invariantes del SAD, que se identifican con el prefijo RNI.
SAD / SDD	<i>Software Architecture Document / Software Design Document</i> .
SRS	<i>Software Requirements Specification</i> . Especificación de requerimientos.
Tamizaje	Conjunto de pruebas que determinan si la sangre donada es apta para transfusión.
Unidad	Ejemplar individual de un componente sanguíneo, trazable de forma única.
Unidad no apta	Unidad que no puede transfundirse y debe seguir el protocolo de disposición final.

Continúa en la página siguiente

Tabla 4 — continuación

Término	Definición
WCAG	<i>Web Content Accessibility Guidelines</i> . Criterios de accesibilidad para contenido web.
YARP	<i>Yet Another Reverse Proxy</i> . Biblioteca de proxy inverso sobre ASP.NET Core, usada como API Gateway.
Zona de confianza	Agrupación de componentes con el mismo nivel de exposición, separada de las demás por una red de contenedores propia. ADR-014 define cinco: borde, presentación, aplicación, datos y observabilidad.

1.5 Referencias

Cuadro 5: Normas, estándares y fuentes que este documento aplica.

Ref.	Fuente
[1]	Congreso de la República de Colombia. Ley 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales.
[2]	Presidencia de la República de Colombia. Decreto 1571 de 1993, por el cual se reglamenta el funcionamiento de bancos de sangre.
[3]	Ministerio de Salud de Colombia. Resolución 901 de 1996 y normas modificatorias.
[4]	ISO/IEC 25010:2023. <i>Systems and software engineering — SQuaRE — Product quality model</i> .
[5]	ISO/IEC/IEEE 29148:2018. <i>Requirements engineering</i> . Estructura y atributos de un requerimiento.
[6]	ISO/IEC/IEEE 42010:2022. <i>Architecture description</i> . Fundamento del uso de vistas y de los ADR.
[7]	ISO/IEC/IEEE 15289:2019. <i>Content of life-cycle information items</i> . Estructura de los documentos del proyecto.
[8]	Schwaber, K. y Sutherland, J. <i>The Scrum Guide</i> , edición 2020.
[9]	Driessen, V. <i>A successful Git branching model</i> (GitFlow), 2010.
[10]	Brown, S. <i>The C4 model for visualising software architecture</i> .
[11]	Kruchten, P. <i>Architectural Blueprints — The 4+1 View Model of Software Architecture</i> . IEEE Software, 1995.
[12]	Evans, E. <i>Domain-Driven Design</i> , 2003. Origen del concepto de contexto acotado.
[13]	Preston-Werner, T. <i>Semantic Versioning 2.0.0</i> .
[14]	<i>Conventional Commits 1.0.0</i> . Convención de mensajes de commit.
[15]	Wiggins, A. <i>The Twelve-Factor App</i> . Configuración por entorno y paridad entre ambientes.

Continúa en la página siguiente

Tabla 5 — continuación

Ref.	Fuente
[16]	W3C. <i>Web Content Accessibility Guidelines (WCAG) 2.2</i> , nivel AA.
[17]	OpenAPI Initiative. <i>OpenAPI Specification 3.1</i> .
[18]	Thoughtworks. <i>Build your own Technology Radar</i> . Método de los cuadrantes y anillos.
[19]	Wake, B. <i>INVEST in Good Stories, and SMART Tasks</i> , 2003.
[20]	OWASP. <i>Application Security Verification Standard y Top Ten</i> . Referencia de los controles de la Sección 7.7.
[21]	VMware Tanzu. <i>Spring Boot support policy and release calendar</i> . Fuente de las fechas de fin de soporte de la Sección 3.3.
[22]	Oracle. <i>Java SE Support Roadmap</i> . Fuente de la ventana de soporte de las versiones LTS de Java.
[23]	Eclipse Adoptium. <i>Temurin Support</i> . Fuente de la ventana de soporte de la distribución Temurin 25.
[24]	Microsoft. <i>.NET and .NET Core Support Policy</i> . Fuente de la ventana de soporte de .NET 10.

2. Restricciones de diseño

La elección de herramientas no es libre. Responde a tres fuentes, en este orden de prelación: las restricciones de diseño impuestas por el cliente, el marco normativo colombiano y las condiciones reales del proyecto. Ninguna herramienta entra al catálogo de la Sección 3 sin que pueda rastrearse hasta al menos una de ellas, y ninguna restricción queda sin instrumento que la satisfaga. Esa doble verificación es el Anexo A.

Las ocho restricciones del cliente, RD-01 a RD-08, son idénticas a las del contexto maestro del proyecto y a las del SRS, y así deben permanecer. Las restricciones normativas y las condiciones del proyecto de esta sección están acotadas a las que imponen una exigencia sobre el instrumental de trabajo; el SRS recoge además las que imponen una exigencia sobre el producto, y el Documento de Proyecto las que acotan el alcance. Ante divergencia en una restricción del cliente, la fuente de verdad es el contexto maestro y este documento se corrige, no al revés.

2.1 Restricciones del cliente

El cliente formuló ocho restricciones de diseño, identificadas con el prefijo RD. Son de obligatorio cumplimiento y no admiten negociación técnica: una alternativa que las incumpla queda descartada aunque sea superior en todo lo demás.

Cuadro 6: Restricciones de diseño impuestas por el cliente (RD-01 a RD-08).

ID	Restricción	Exigencia técnica derivada
RD-01	La solución debe ser distribuida. Queda explícitamente prohibido el monolito modular. Distribuido no equivale a microservicios: el equipo debe sustentar el estilo que elija.	Servicios desplegados de forma independiente, comunicados por contrato y sin base de datos compartida. Obliga a justificar la granularidad elegida mediante ADR.
RD-02	El stack incluye como mínimo un componente en Java y como mínimo uno en .NET.	Dos plataformas de ejecución simultáneas, cada una con su cadena de construcción, su marco de pruebas y su instrumentación de métricas. Duplica el esfuerzo de integración continua.
RD-03	Todo contenerizado.	Una imagen de contenedor por componente ejecutable, orquestación local reproducible y registro de imágenes. Excluye toda plataforma de despliegue que no acepte una imagen propia.
RD-04	Documentación técnica en Markdown con Mermaid.	Los diagramas viven en el repositorio como texto versionable, no como imágenes exportadas. Excluye las herramientas de diagramación con formato propio.
RD-05	Control de versiones con GitFlow.	Ramas de larga vida, ramas de publicación y de corrección urgente, con reglas de fusión explícitas y protección de rama configurada.
RD-06	Múltiples repositorios bajo una organización de GitHub.	Versionado independiente por repositorio y trazabilidad de una historia que toca varios de ellos.
RD-07	Todo el trabajo, técnico y funcional, va al backlog.	Herramienta de backlog capaz de contener historias técnicas, de infraestructura y de gestión, no solo funcionalidad de usuario.
RD-08	La seguridad es atributo de calidad de primer orden: el sistema maneja datos personales y sensibles.	Autenticación y autorización centralizadas en el borde, filtrado fino por jurisdicción en cada servicio, bitácora de auditoría, ausencia de secretos en el repositorio y plataformas de ejecución con soporte de seguridad vigente.

2.2 Restricciones normativas y del dominio

Cuadro 7: Restricciones normativas y del dominio (RN-01 a RN-04).

ID	Norma	Restricción que impone sobre las herramientas
RN-01	Ley 1581 de 2012 [1]	Los datos de salud son sensibles. Ninguna herramienta puede almacenar, registrar ni transmitir la causa clínica de una unidad no apta. Condiciona qué se escribe en las bitácoras, qué se envía a la recolección de métricas y qué puede desagregar un tablero.
RN-02	Decreto 1571 de 1993 [2]	La donación no puede remunerarse. Restringe el subsistema de reconocimiento a mecánicas sin contraprestación económica, directa ni indirecta. Ninguna integración con pasarelas de pago, monederos o programas de puntos canjeables entra al catálogo.
RN-03	Resolución 901 de 1996 y modificatorias [3]	Define captación, tamizaje, almacenamiento y disposición final. El modelo de datos y sus migraciones deben reflejar esos estados sin pérdida de trazabilidad, lo que exige base de datos transaccional y migraciones versionadas y reversibles.
RN-04	ISO/IEC 25010:2023 [4]	Marco de referencia de los atributos de calidad. Se adopta la revisión de 2023, con nueve características, no la de 2011 con ocho. Incorpora la capacidad de interacción, que es la que sustenta el criterio de accesibilidad.

2.3 Condiciones del proyecto

Cuadro 8: Condiciones del proyecto que acotan la selección (CP-01 a CP-04).

ID	Condición	Consecuencia sobre la selección
CP-01	Horizonte de cinco sprints de dos semanas	Tecnologías conocidas por el equipo o de curva de aprendizaje corta. Se descarta toda herramienta que exija operar infraestructura adicional.
CP-02	Equipo de siete integrantes de perfil junior, con nueve horas semanales cada uno	Se prefiere una pieza autónoma sobre una pieza potente que requiera un plano de control, una base de datos propia o un operador dedicado. Capacidad estimada: 126 horas por sprint para todo el equipo.

Continúa en la página siguiente

Tabla 8 — continuación

ID	Condición	Consecuencia sobre la selección
CP-03	Presupuesto de desembolso cero	Toda herramienta en nivel gratuito, licencia de código abierto o licencia educativa. Se verifica que la edición gratuita conserve las funciones que el proyecto necesita y que sus cuotas alcancen para el volumen previsto; no basta con que exista. Los programas educativos a los que el equipo tiene derecho por su condición de estudiantes cuentan como recurso legítimo bajo esta condición, siempre que su vigencia cubra el horizonte del proyecto.
CP-04	Datos exclusivamente sintéticos	No se requieren controles de anonimización ni herramientas de enmascaramiento. A cambio, se exige que el origen sintético sea evidente en la propia demostración.

2.4 Regla de precedencia entre restricciones

Cuando dos restricciones apuntan en direcciones distintas, se resuelve en este orden y se deja constancia del descarte:

1. **RN sobre todo lo demás.** Ninguna conveniencia técnica ni de plazo justifica incumplir la ley.
2. **RD sobre CP.** Una restricción del cliente no se relaja porque sea incómoda para el equipo; si resulta inviable, se escala al cliente por el Product Owner, no se ignora.
3. **CP como criterio de desempate** entre alternativas que ya cumplen RN y RD.

Esta regla se aplica dos veces en el catálogo. Una plataforma de despliegue que no acepte una imagen de contenedor propia incumple RD-03 y queda descartada, por cómoda que resulte: por esa razón Vercel está en anillo **Evitar** en la entrada 43. Y una plataforma de ejecución sin parches de seguridad vigentes incumple RD-08, que declara la seguridad como atributo de primer orden: por esa razón la línea 3.x de Spring Boot queda fuera del catálogo pese a ser la más conocida por el equipo. Una restricción del cliente no se somete a evaluación comparativa; se cumple.

3. Tech Radar

3.1 Metodología

El Tech Radar clasifica cada entrada en dos dimensiones: el **cuadrante**, que indica a qué categoría pertenece, y el **anillo**, que indica qué nivel de compromiso tiene el equipo con ella. El método es el propuesto por Thoughtworks [18], adaptado a un proyecto de alcance cerrado.

Cuadro 9: Significado de los cuatro anillos.

Anillo	Significado
Adoptar	Decisión tomada. Es el estándar del proyecto y se usa sin requerir justificación adicional. Aparecerá en el SAD y en el SDD como tecnología del sistema.
Probar	Se usará en un alcance acotado durante el proyecto. Se evalúa su desempeño antes de generalizar. La promoción o el retiro se registran en el control de versiones de este documento.
Evaluar	De interés, pero sin compromiso. Solo se incorpora si un requerimiento lo hace necesario, y mediante ADR. No se estudia por curiosidad: cada entrada de este anillo tiene escrito el disparador concreto que la activaría, y ese disparador debe ser alcanzable sin violar una restricción de la Sección 2.
Evitar	Descartado de forma explícita para este proyecto, con justificación documentada. Aparece en el radar precisamente para que nadie lo reintroduzca por desconocimiento.

Los cuatro cuadrantes son: Lenguajes y Frameworks, Datos e Infraestructura, Gestión y Colaboración, y Técnicas y Prácticas.

Regla de ubicación. Una entrada del radar nombra una tecnología o una práctica, no una decisión puntual. Si lo que se quiere registrar es a quién le corresponde una responsabilidad dentro del sistema, eso es un ADR, no una entrada de catálogo. Los lenguajes, marcos, bibliotecas y cadenas de construcción van al primer cuadrante, con independencia de qué función cumplan en la arquitectura.

Regla de agrupación. Una entrada puede reunir varias herramientas cuando comparten una única decisión y una única justificación, por ejemplo las migraciones versionadas de las dos plataformas, o las dos herramientas comerciales del mismo proveedor que se descartan por el mismo motivo. Cuando la decisión sobre cada herramienta sería distinta, se separan en entradas independientes.

Regla de vigencia. Ninguna entrada en anillo **Adoptar** puede estar fuera de su ventana de soporte de seguridad. Es la aplicación directa de RD-08 sobre el instrumental, y se verifica al cierre de cada sprint junto con el registro de riesgos. Una tecnología correctamente elegida hace seis meses puede haber dejado de serlo hoy, y el documento no lo detecta solo.

Regla de numeración. El número de una entrada es estable: no cambia cuando la entrada cambia de anillo, y las entradas nuevas toman el siguiente número disponible dentro del cuadrante que les corresponde. Los ADR y los demás documentos citan las entradas por número, y reenumerar el catálogo rompería esas referencias.

Regla de movimiento. Una entrada solo cambia de anillo mediante una nota en el control de versiones de este documento, y solo después de que la Mesa de Arquitectura lo haya acordado. Una entrada que llega a **Evitar** no se elimina del catálogo: permanece con su justificación, porque el valor de un radar está tanto en lo que descarta como en lo que adopta.

Regla de incorporación. Toda herramienta que no esté en el catálogo y que alguien necesite usar exige, como mínimo, una entrada nueva aprobada en Mesa de Arquitectura; si tiene consecuencias estructurales, además un ADR. No se incorpora una dependencia y se documenta después.

3.2 Radar visual

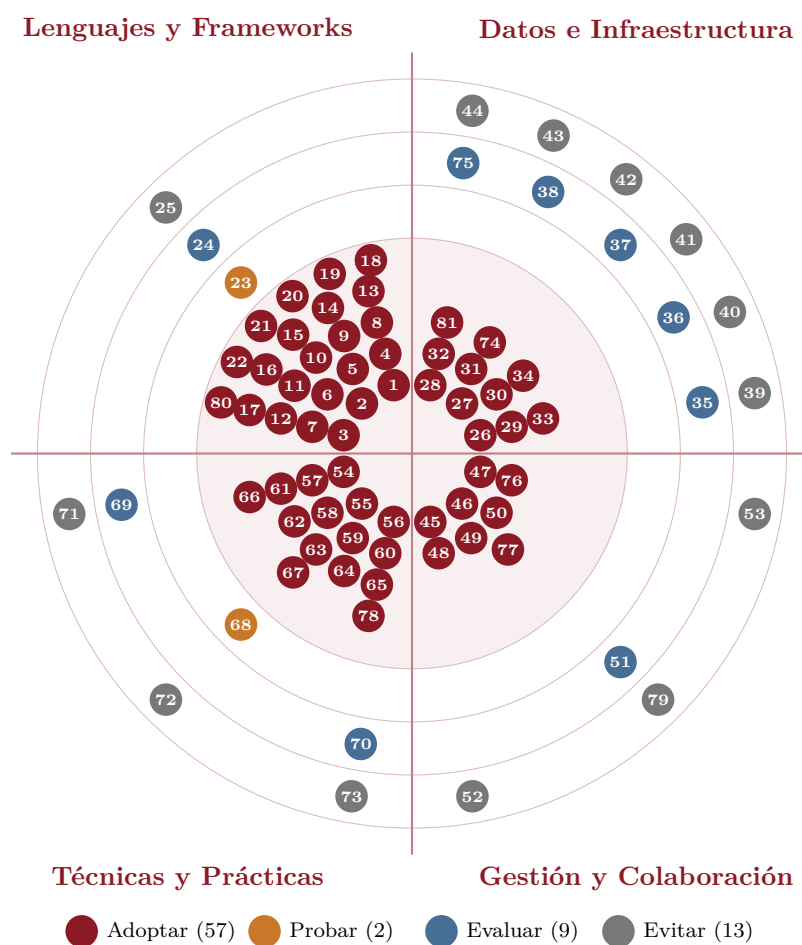


Figura 1: Tech Radar de RedVital, versión 3.0. Cada punto es una entrada del catálogo; la numeración corresponde a las tablas de las secciones 3.3 a 3.6. Cuanto más cerca del centro, mayor es el compromiso del equipo con esa entrada.

3.3 Cuadrante 1 — Lenguajes y Frameworks

Cuadro 10: Catálogo del cuadrante Lenguajes y Frameworks (entradas 1 a 25 y 80).

#	Herramienta	Anillo	Descripción y uso	Justificación
1	Java 25 (LTS), distribución Eclipse Temu- rin	Adoptar	Lenguaje del Servicio de Donación —donantes, donaciones, unidades, trazabilidad, inventario y transferencias— y del trabajador de Notificaciones.	Registrado en ADR-009. Es la versión de soporte extendido vigente: Oracle declara soporte principal hasta septiembre de 2030, y la distribución Eclipse Temurin, gratuita y de código abierto, publica actualizaciones hasta septiembre de 2031. La imagen de construcción es la oficial de Temurin 25 con su JDK, fijada a la versión de actualización completa; la de ejecución lleva un entorno reducido generado con <code>jlink</code> desde ese mismo JDK sobre una base <i>distroless</i> (entrada 81). Java es además la tecnología con mayor dominio en el equipo (CP-01), y su tipado estático reduce errores en un dominio donde la trazabilidad de cada unidad es crítica. Satisface la mitad Java de RD-02.

Continúa en la página siguiente

Tabla 10 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
2	Spring Boot 4.1	Adoptar	Framework del Servicio de Donación y del trabajador de Notificaciones. Expone la API REST, gestiona la inyección de dependencias y la configuración.	Registrado en ADR-009. Es la línea con soporte de código abierto vigente, hasta el 31 de julio de 2027, compatible con Java 17 a 26. La línea 3.x completa salió de soporte el 30 de junio de 2026, de modo que adoptarla significaría construir sobre una base sin parches de seguridad, en contradicción directa con RD-08. Como el servicio todavía no existe, el costo de arrancar aquí es nulo; el costo asumido es conocer las convenciones de Spring Framework 7 y Jackson 3 desde el primer commit.
3	Spring Data JPA / Hibernate	Adoptar	Mapeo objeto-relacional del Servicio de Donación.	Su soporte de herencia y polimorfismo habilita el modelo de dominio genérico que permite incorporar otros tipos de donación sin rehacer trazabilidad ni inventario (RNF-05 y RI-01). La versión queda determinada por la línea de la entrada 2, no se elige por separado.
4	Spring Security	Adoptar	Validación del token de acceso y autorización dentro del Servicio de Donación, como servidor de recursos: firma RS256 contra las claves públicas del Servicio de Identidad, emisor, audiencia y vencimiento (ADR-008).	El gateway resuelve la compuerta; el servicio resuelve el filtrado fino por jurisdicción a nivel de registro. RNF-02 exige los dos niveles, no solo uno: el gateway no puede saber a qué institución pertenece una unidad concreta.

Continúa en la página siguiente

Tabla 10 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
5	Spring Scheduler	Adoptar	Tareas programadas de ejecución periódica en el Servicio de Donación.	Cubre RNF-08, retiro automático de unidades vencidas; RF-15, expiración de reservas; y el ciclo del escalamiento de ADR-013, que se dispara al crearse una alerta de escasez y se repite cada quince minutos. No agrega infraestructura externa, coherente con CP-02.
6	Maven	Adoptar	Cadena de construcción y gestión de dependencias del Servicio de Donación.	Sin cadena de construcción declarada, el lenguaje y el marco adoptados no se ejecutan. Se elige frente a Gradle porque su archivo de construcción es declarativo y legible sin conocer un lenguaje adicional, lo que importa con siete integrantes de perfil junior (CP-02), y porque es la convención mayoritaria del ecosistema Spring.
7	.NET 10 (LTS) / C#	Adoptar	Lenguaje de los servicios de Identidad, Institucional y Campañas, y del API Gateway.	Versión de soporte extendido publicada en noviembre de 2025, con soporte hasta el 10 de noviembre de 2028. Satisface la mitad .NET de RD-02 y cubre con holgura la vigencia del proyecto. Los cuatro componentes .NET parten de una plantilla común de servicio, con autenticación, salud, métricas y correlación, que reduce el riesgo de aprendizaje sobre la plataforma que el equipo domina menos (ADR-014).

Continúa en la página siguiente

Tabla 10 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
8	ASP.NET Core 10	Adoptar	Framework de los servicios de Identidad, Institucional y Campañas, y anfitrión del API Gateway.	Su modelo de autorización basado en políticas y requisitos personalizados es la forma directa de expresar la regla territorial jerárquica de RNF-02 en un solo lugar. Incluye de serie la validación de tokens JWT contra claves públicas y el componente de limitación de tasa que usa el gateway (ADR-007, ADR-008).
9	Entity Framework Core 10	Adoptar	Mapeo objeto-relacional y migraciones de los servicios de Identidad, Institucional y Campañas.	Opción por defecto del ecosistema .NET. Integra el versionado del esquema en el mismo flujo de trabajo del código.
10	SDK de .NET y NuGet	Adoptar	Cadena de construcción y gestión de dependencias de los tres servicios .NET y del gateway.	Misma razón que la entrada 6. A diferencia del caso de Java, no hubo alternativa que evaluar: es la cadena única de la plataforma. Se fija la versión mayor del SDK para que la construcción sea reproducible dentro del contenedor.

Continúa en la página siguiente

Tabla 10 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
11	YARP	Adoptar	Biblioteca del API Gateway. Enrutamiento hacia los cuatro servicios con rutas publicadas, validación del token, compuerta por ruta y rol, limitación de tasa en dos escalas —anónima por origen y autenticada por sesión— y entrega de las denegaciones al Servicio de Identidad.	Registrado en ADR-007. Es gratuita y de licencia permisiva, se configura por archivo o por código, y corre como una sola pieza sin base de datos ni plano de control propios (CP-02). A diferencia de las opciones declarativas, permite escribir en un solo lugar la compuerta de autorización y el registro de denegaciones, que son el requisito de mayor peso bajo RD-08. Mantiene el stack en dos lenguajes en lugar de introducir un tercero. Nunca enruta rutas con prefijo <code>/internal/</code> .
12	Instrumentación de métricas: Micrometer con Actuator, y OpenTelemetry para .NET	Adoptar	Exposición de métricas de cada servicio para su recolección.	Micrometer es el estándar en Spring y expone el punto de recolección sin código adicional. En .NET se prefiere OpenTelemetry sobre una biblioteca específica de un recolector, porque no ata la instrumentación a un único destino.
13	TypeScript 5.7	Adoptar	Lenguaje de la Aplicación Web.	Añade tipado a JavaScript, lo que reduce errores de integración contra los contratos de cuatro servicios. La configuración estricta hace que un import sobrante rompa la construcción en vez de pasar desapercibido.
14	React 19	Adoptar	Biblioteca de interfaz. Construye los paneles de donante, banco de sangre y entidad territorial.	Experiencia previa del equipo (CP-01) y modelo de componentes que permite reutilizar elementos entre los siete perfiles de usuario.

Continúa en la página siguiente

Tabla 10 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
15	Vite 6	Adoptar	Servidor de desarrollo y empaquetador de la Aplicación Web.	React y TypeScript no se ejecutan por sí solos en un navegador: hay que compilar JSX y resolver módulos. Vite es el mínimo necesario para que lo aprobado funcione. Registrado en ADR-006.
16	React Router 7	Adoptar	Enrutamiento de la Aplicación Web, con rutas anidadas bajo un contenedor de aplicación y rutas paramétricas.	Sin enrutamiento no hay navegación diferenciada por rol, que es el mecanismo con el que se demuestra RNF-02 en la interfaz: una entrada de menú que no corresponde al rol no se dibuja, no se deshabilita. Registrado en ADR-006.
17	Tailwind CSS 4	Adoptar	Estilos por utilidades. La configuración vive en CSS, dentro del bloque de tema, junto con color, tipografía, sombras y animaciones.	Se validó sobre los tres módulos del prototipo de alta fidelidad antes de generalizarse, y el resultado se registró en el ADR-006. Permite construir interfaces consistentes sin mantener hojas de estilo paralelas.
18	Radix UI (primitivos)	Adoptar	Primitivos accesibles sin estilos propios. En uso hoy para el componente de pestañas.	Aporta manejo de teclado y atributos de accesibilidad correctos sin imponer identidad visual, de modo que el vinotinto institucional no compite contra un tema ajeno. Contribuye a la entrada 66.
19	Node.js 24 (LTS) y npm	Adoptar	Entorno de ejecución de la cadena de construcción de la Aplicación Web.	Se fija la versión mayor para que la construcción sea reproducible entre los equipos de los siete integrantes y dentro del contenedor.

Continúa en la página siguiente

Tabla 10 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
20	Análisis estático: Checkstyle y SpotBugs, analizadores de .NET, ESLint	Adoptar	Verificación automática de estilo y de defectos comunes, ejecutada en cada Pull Request.	La política de integración continua de la Sección 7.5 exige análisis estático; sin esta entrada exigiría una capacidad que el catálogo no provee. Se eligen los analizadores nativos de cada plataforma porque no requieren servicio externo ni cuenta adicional (CP-03).
21	Verificación de accesibilidad: complemento de accesibilidad para ESLint y auditoría de navegador	Adoptar	Detección automática de defectos de accesibilidad en el código de la interfaz y auditoría de la página construida.	La entrada 66 y la definición de terminado exigen verificar contraste, foco visible y navegación por teclado; sin instrumento, esa verificación depende de que alguien se acuerde. El costo marginal es nulo: el complemento es una regla más sobre el analizador que la entrada 20 ya adopta.
22	Marcos de prueba: JUnit 5 con Mockito, xUnit y Vitest	Adoptar	Pruebas unitarias y de integración en Java, .NET y la Aplicación Web respectivamente.	Son las opciones por defecto de sus tres ecosistemas y no hay alternativa en evaluación, de modo que la decisión está tomada. Lo que sigue sin fijarse es la estrategia de cobertura, y eso vive en la entrada 68.

Continúa en la página siguiente

Tabla 10 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
23	Ejecución asíncrona con reintento (Spring)	Probar	Entrega de notificaciones en el trabajador de Notificaciones, con reintento de espera creciente, fuera del proceso transaccional del Servicio de Donación.	Cubre RF-12 sin desplegar un intermediario de mensajería, lo que respecta CP-02. ADR-014 fija que el trabajador consume la bandeja de salida del Servicio de Donación (entrada 78). Permanece en <i>Probar</i> hasta que el trabajador drene en QA la acumulación de quinientos eventos de EC-11 dentro de su plazo; si no lo consigue, se escala a la entrada 38.
24	Generación del cliente TypeScript desde el contrato OpenAPI	Evaluar	Derivación automática de los tipos y las funciones de llamada de la Aplicación Web desde la especificación de cada servicio.	Disparador: que aparezca la primera desincronización entre un contrato y el código de la Aplicación Web. Elimina la transcripción manual de tipos entre repositorios, pero añade un paso de generación a la construcción y por eso no se adopta por defecto.
25	Suites de interfaz con tema propio y gestores de estado global (MUI, Ant Design, Redux)	Evitar	Bibliotecas de componentes con identidad visual incorporada y almacenes de estado centralizados.	Descartado en ADR-006. Una suite con tema propio obliga a pelear contra su diseño para imponer la identidad institucional, y su semántica de color choca con la decisión de marca de usar un vinotinto que no debe leerse como error del sistema. Un gestor de estado global no se justifica cuando el estado compartido se reduce a la sesión.

Continúa en la página siguiente

Tabla 10 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
80	Gatling	Adoptar	Generación de carga para las pruebas de carga y de sobrecarga de EC-07, EC-08, EC-15, EC-16 y EC-26, ejecutada desde el equipo de un integrante contra la pila de medición. Los escenarios de carga se escriben en Java y viven en redvital-infra , junto a la composición que miden.	Las medidas de esos escenarios exigen usuarios concurrentes sostenidos, rampas y percentiles de latencia; sin un instrumento, la carga se improvisaría y el resultado no sería repetible. Se elige Gatling porque su edición comunitaria es gratuita y de código abierto (CP-03), sus simulaciones se escriben en Java, que el equipo ya domina (CP-01), y se ejecuta con Maven sin servicio externo. El generador corre fuera de la máquina de QA para no competir por los recursos que se miden.

3.4 Cuadrante 2 — Datos e Infraestructura

Cuadro 11: Catálogo del cuadrante Datos e Infraestructura (entradas 26 a 44, 74, 75 y 81).

#	Herramienta	Anillo	Descripción y uso	Justificación
26	PostgreSQL 16	Adoptar	Base de datos relacional. Cuatro instancias, una por cada servicio con base: Identidad, Institucional, Campañas y Donación.	El dominio exige garantías transaccionales: una unidad de sangre no puede quedar simultáneamente disponible y desechada (RN-03). La separación por servicio materializa RD-01 en la capa de datos. Cada base tiene tres roles: el superusuario, reservado al guion de inicialización; el propietario, con el que migran, respaldan y restauran los trabajos de la base; y el de servicio, sin definición de esquema y con solo inserción y lectura sobre las tablas de solo anexado. El respaldo diario usa <code>pg_dump</code> de la misma imagen oficial, de modo que no entra herramienta nueva. Sobre la versión: PostgreSQL da cinco años de soporte a cada versión mayor, y la 16 los tiene hasta noviembre de 2028, de modo que cumple la regla de vigencia sin necesidad de estar en la última. Registrado en ADR-005 y ADR-014.

Continúa en la página siguiente

Tabla 11 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
27	Docker con imágenes multi-etapa	Adoptar	Empaquetado de todo componente ejecutable: los cinco servicios, el gateway, la Aplicación Web, el proxy de borde, los cuatro trabajos de migración y los de respaldo y restauración.	RD-03 lo exige. La construcción multi-etapa deja imágenes de ejecución sin herramientas de compilación ni intérprete de órdenes (entrada 81), lo que reduce la superficie de ataque y es relevante bajo RD-08. Como esas imágenes no tienen intérprete, su verificación de salud se declara en forma de ejecución directa de un programa de sonda incluido en la imagen, nunca como orden de intérprete.
28	Docker Compose	Adoptar	Orquestación del sistema completo en Desarrollo y en QA.	Levanta con un solo comando los doce contenedores de Desarrollo y los catorce de QA, más los cuatro trabajos de migración, con el mismo archivo en ambos ambientes y las diferencias expresadas solo como variables. Es la forma más barata de cumplir la paridad de entornos de la entrada 60. El arranque se encadena por condición de salud, en el orden que fija ADR-014. El atributo de versión en la raíz del archivo está obsoleto en la versión vigente de la herramienta y debe omitirse.

Continúa en la página siguiente

Tabla 11 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
29	Tokens JWT con publicación de claves (JWKS)	Adoptar	Formato del token de acceso, firmado con RS256 y claves RSA de 3072 bits, que transporta ocho reivindicaciones sin dato personal: identidad, rol, jurisdicción, emisor, audiencia, emisión, vencimiento e identificador del token. El Servicio de Identidad publica sus claves públicas, y el gateway y los servicios validan sin consultarlo en cada petición: con el esquema JWT Bearer de ASP.NET Core 10 en el lado .NET y con Spring Security como servidor de recursos en el lado Java.	RD-08 exige autenticación centralizada, y eso obliga a fijar no solo quién valida el token sino con qué mecanismo. La publicación de claves evita que la validación se convierta en una llamada entre servicios en cada petición, que sería una dependencia en tiempo de ejecución contraria a la independencia que busca RD-01. Registrado en ADR-008: emisor único en el Servicio de Identidad, token de acceso de quince minutos, sesión de renovación de ocho horas y rotación de la clave de firma en cada sprint sin invalidar sesiones.

Continúa en la página siguiente

Tabla 11 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
30	Migraciones versionadas: Flyway y EF Core Migrations	Adoptar	Versionado y aplicación del esquema, ejecutados por un trabajo de migración efímero por base, que carga además la porción del conjunto sintético que le corresponde. Flyway en el Servicio de Donación, EF Core Migrations en los servicios .NET.	Permite que los cambios de esquema viajen con el código y sean reproducibles, en lugar de aplicarse a mano sobre un ambiente ya desplegado. Separar la migración del arranque del servicio hace que un fallo de esquema sea distinguible de un fallo de ejecución. Toda migración se escribe en dos tiempos, expansión y contracción: una versión solo añade estructura compatible con la versión anterior del servicio, y lo que deja de usarse se elimina en una versión posterior. Así, revertir un despliegue nunca exige deshacer una migración. RN-03 exige además que los estados del ciclo de la unidad sean reconstruibles, de modo que ninguna migración, ni siquiera la de contracción, elimina eventos de trazabilidad.
31	Prometheus	Adoptar	Recolección y almacenamiento de las métricas del gateway y de los cinco servicios, en QA. Raspa únicamente el puerto de gestión 8081 de cada componente, y solo los destinos declarados por nombre en su configuración.	Estándar de facto, gratuito y desplegable como un contenedor más. Es el instrumento con el que se verifica de forma objetiva RNF-04, el tiempo de respuesta de la consulta de inventario, y con el que se leen las mediciones de carga de la entrada 80. No tiene ruta en el proxy de borde: solo lo alcanza Grafana.

Continúa en la página siguiente

Tabla 11 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
32	Grafana	Adoptar	Tableros de operación y alertas sobre las métricas recolectadas, en el ambiente de QA.	Existen tableros publicados para Spring Boot y para .NET, lo que evita construirlos desde cero. Solo se alcanza a través del proxy de borde, en la ruta <code>/grafana/</code> , con usuarios nominales y tableros aprovisionados desde el repositorio en modo de solo lectura. Restricción de uso derivada de RN-01: ningún tablero puede desagregar información hasta un donante identificable ni exponer la causa clínica de una unidad.
33	GitHub Container Registry	Adoptar	Registro de las imágenes producidas por la integración continua.	Incluido en la organización de GitHub que ya exige RD-06. Evita introducir un proveedor adicional solo para almacenar imágenes. Su cuota gratuita en repositorios privados es limitada, lo que condiciona la política de publicación de la Sección 7.5.

Continúa en la página siguiente

Tabla 11 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
34	Máquina virtual institucional con Docker Compose	Adoptar	Anfitrión del ambiente de QA, que es también el de demostración. Un único servidor Linux que ejecuta los catorce contenedores de QA con el mismo archivo de orquestación de la entrada 28.	Es la opción que mejor cumple las tres restricciones a la vez: acepta imágenes propias (RD-03), no tiene costo para el equipo (CP-03) y no introduce ninguna herramienta nueva que aprender, porque es el mismo Compose que ya se usa en local (CP-01). Su memoria se dimensiona sobre el consumo en reposo de los catorce contenedores medido en QA, con un margen del cincuenta por ciento para la carga de EC-16. Condición de viabilidad, registrada como riesgo RT-02: la máquina debe ser alcanzable desde fuera de la red institucional, o el cliente no podrá abrir la demostración.
35	Agregación de registros (Loki con recolector)	Evaluar	Recolección centralizada de la salida de los contenedores, consultable desde el mismo Grafana de la entrada 32.	Disparador: el primer defecto cuya comprensión exija correlacionar los registros de dos o más servicios y que el identificador de correlación, buscado contenedor por contenedor, no resuelva en el plazo de EC-20. No se adopta por defecto porque añadiría dos contenedores más sobre una máquina que ya sostiene catorce, y CP-02 pide preferir la pieza autónoma.

Continúa en la página siguiente

Tabla 11 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
36	Keycloak	Evaluar	Proveedor de identidad completo, con federación, gestión de usuarios y emisión de tokens estándar.	Disparador: que el cliente exija federación con un proveedor externo o gestión de credenciales de nivel productivo. Es la opción correcta para un sistema real, pero añade un contenedor con base de datos propia y un modelo de configuración que el equipo no domina, lo que choca con CP-01 y CP-02 frente a la entrada 29.
37	Redis	Evaluar	Almacenamiento en memoria. Bloqueos distribuidos y expiración automática de claves.	Disparador: que la concurrencia de la reserva de cupo (RF-15) no se resolviera satisfactoriamente en base de datos.
38	RabbitMQ	Evaluar	Intermediario de mensajería para comunicación asíncrona entre servicios.	Disparador: que la bandeja de salida expuesta por contrato (entrada 78) no sostenga la medida de EC-11. No se adopta por defecto porque añadiría una pieza de infraestructura que el equipo no tiene experiencia en operar (CP-02).

Continúa en la página siguiente

Tabla 11 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
39	Render y Railway	Evitar	Plataformas de despliegue gestionado evaluadas y descartadas.	Render limita cada cuenta a una sola base de datos gratuita, y RedVital necesita cuatro instancias independientes por decisión estructural de ADR-005 y ADR-014; además esa base expira a los treinta días de creada, los servicios gratuitos se suspenden tras quince minutos sin tráfico, y el conjunto de la cuenta dispone de 750 horas de instancia al mes, que catorce contenedores encendidos consumen en menos de tres días. Railway no tiene nivel gratuito permanente: entrega un crédito único de prueba y luego factura. Ninguna de las dos es viable bajo CP-03, y la incompatibilidad es estructural, no de margen.
40	Traefik	Evitar	Proxy inverso con descubrimiento automático de contenedores.	Sería candidato si el gateway se redujera a enrutar, pero eso no puede ocurrir sin incumplir RD-08, que exige autorización centralizada en el borde. A esto se suma que su validación de token y su integración con proveedores de identidad pertenecen a la edición comercial, de modo que la autorización tendría que delegarse a un servicio externo. Tampoco se adopta como proxy de borde: ADR-015 elige la entrada 74.

Continúa en la página siguiente

Tabla 11 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
41	Kong Gateway (edición libre) y Ocelot	Evitar	Alternativas de API Gateway evaluadas y descartadas.	Kong dejó de publicar imágenes de contenedor preconstruidas de su edición libre a partir de la versión 3.10, lo que obliga a compilar desde fuente o a congelarse en una versión sin parches: incompatible con RD-03 y con CP-03. Ocelot arrastra un proceso de deprecación de opciones y perdió su cadena de integración continua histórica; YARP es la apuesta sostenida de la misma plataforma.
42	Kubernetes	Evitar	Orquestador de contenedores en clúster.	Descartado: RD-03 exige contenerización, no orquestación en clúster. El costo de aprender y operar un clúster no es asumible bajo CP-01 y CP-02.
43	Vercel	Evitar	Plataforma de despliegue especializada en aplicaciones de frontend.	Su modelo de despliegue no parte de una imagen de contenedor propia, de modo que servir allí la Aplicación Web dejaría fuera de RD-03 a uno de los componentes del sistema. Con la entrada 34, además, no tendría función: el frontend se sirve desde el mismo servidor que el resto.
44	Bases de datos NoSQL documentales	Evitar	Almacenamiento no relacional orientado a documentos.	Descartado en ADR-005: el modelo de trazabilidad es fuertemente relacional y requiere transacciones. Adoptarlo introduciría riesgo de inconsistencia sin beneficio para este dominio.

Continúa en la página siguiente

Tabla 11 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
74	Caddy 2	Adoptar	Proxy de borde de la Zona 1: terminación TLS, redirección de HTTP a HTTPS, cabeceras de seguridad, tamaño máximo de cuerpo de 1 MiB y reparto hacia la Aplicación Web, el gateway y, en QA, Grafana. Único contenedor con puerto publicado al anfitrión; escucha en los puertos no privilegiados 8443 y 8080, publicados como 443 y 80, y conserva su autoridad interna en un volumen persistente. La misma imagen sirve además los recursos estáticos de la Aplicación Web, en un contenedor aparte y sin la configuración de borde.	Registrado en ADR-015. Gestiona los certificados por sí mismo, con autoridad interna en Desarrollo y QA y por ACME en Producción, con el mismo archivo de configuración en los tres ambientes. Redacta por defecto credenciales y cookies en sus registros de acceso. Se usa la imagen oficial, fijada por versión mayor y menor, sin módulos adicionales: no limita la tasa, que es función de la entrada 11.
75	nginx	Evaluar	Proxy inverso alternativo para la Zona 1.	Disparador: que el borde necesite limitación de tasa propia o balanceo entre réplicas que Caddy estándar no resuelva (ADR-015). Frente a la entrada 74 exige un proceso adicional de emisión y renovación de certificados en cada ambiente, lo que contraría CP-01.

Continúa en la página siguiente

Tabla 11 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
81	Imágenes de ejecución mínimas: ASP.NET Core <i>chiseled</i> y <i>distroless</i> con entorno Java reducido	Adoptar	Base de la etapa de ejecución de las seis imágenes que construye el equipo. El gateway y los tres servicios .NET usan ASP.NET Core 10 en su variante <i>chiseled</i> sobre Ubuntu 24.04. El Servicio de Donación y el trabajador de Notificaciones usan <i>distroless</i> base de Debian 13 en su variante no privilegiada, con un entorno de Temurin 25 reducido con <i>jlink</i> a los módulos que el servicio usa.	RD-08 y la política de la Sección 7.4 exigen que la etapa de ejecución no tenga intérprete de órdenes ni gestor de paquetes, y la imagen oficial de ejecución de Temurin sí los trae, de modo que no cumple la política por sí sola. Las dos bases elegidas no tienen ninguno de los dos, corren con usuario no privilegiado por defecto y reducen el número de paquetes que el escáner de la entrada 76 tiene que vigilar. Consecuencia aceptada: sin intérprete, la verificación de salud se hace con un programa de sonda propio y la depuración, con un contenedor auxiliar que comparte el espacio de procesos. Registrado en ADR-009.

3.5 Cuadrante 3 — Gestión y Colaboración

Cuadro 12: Catálogo del cuadrante Gestión y Colaboración (entradas 45 a 53, 76, 77 y 79).

#	Herramienta	Anillo	Descripción y uso	Justificación
45	GitHub: organización, repositorios, Issues y Projects	Adoptar	Alojamiento del código en múltiples repositorios, gestión del backlog y revisión por Pull Request.	RD-06 exige la organización multi-repositorio y RD-07 exige que todo el trabajo esté en el backlog. Concentrar código y backlog en la misma plataforma hace que la trazabilidad entre historia, rama y fusión sea automática en lugar de manual.

Continúa en la página siguiente

Tabla 12 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
46	GitHub Actions	Adoptar	Integración continua y publicación de imágenes en cada repositorio.	Incluido en el nivel gratuito de la organización. Se define un flujo por repositorio, coherente con la independencia de despliegue que exige RD-01. Sus minutos gratuitos en repositorios privados son finitos, lo que condiciona la política de la Sección 7.5 y el riesgo RT-01.
47	Alertas de dependencias vulnerables y actualización automatizada	Adoptar	Revisión continua de las dependencias declaradas en los nueve repositorios, con propuesta automática de actualización.	RD-08 obliga a tratar la seguridad como atributo de primer orden, y la mayor superficie de riesgo de un proyecto de este tamaño está en sus dependencias, no en su código. Es gratuito en la plataforma ya adoptada y no requiere operar nada.
48	Markdown con Mermaid	Adoptar	Documentación técnica y todos los diagramas de arquitectura, dentro del repositorio de documentación.	RD-04 lo exige. La plataforma renderiza Mermaid de forma nativa, de modo que el diagrama se lee en el navegador y se versiona como texto: un cambio de arquitectura aparece en el historial como una diferencia legible, no como una imagen reemplazada.
49	Overleaf (L ^A T _E X)	Adoptar	Redacción colaborativa de los documentos formales entregables al cliente.	Edición simultánea y formato consistente entre entregas. Convive con la entrada 48 sin contradecirla: Markdown para la documentación técnica viva, L ^A T _E X para el artefacto formal fechado y firmado.
50	Figma	Adoptar	Diseño de interfaces y mockups.	Licencia educativa gratuita (CP-03). Su alcance está acotado: los diagramas técnicos no se hacen aquí, sino en Mermaid, por RD-04.

Continúa en la página siguiente

Tabla 12 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
51	MkDocs Material con GitHub Pages	Evaluar	Publicación de la documentación Markdown como sitio navegable.	Disparador: que el volumen de documentación haga incómoda su lectura en el explorador del repositorio. Renderiza Mermaid y se publica sin costo. No se adopta por defecto porque no aporta al producto y consume capacidad (CP-02).
52	Jira y Confluence	Evitar	Gestión de backlog y base de conocimiento comerciales.	Descartados. Mantener el backlog fuera de la plataforma de código obliga a sincronizar a mano la relación entre historia, rama y Pull Request, que es justamente la trazabilidad que el proyecto debe demostrar. Su presupuesto de licencias deja de ser necesario, lo que obliga a corregir el rubro correspondiente del Documento de Proyecto.
53	Draw.io y Structurizr	Evitar	Herramientas de diagramación técnica alternativas.	Descartadas por conflicto con RD-04. Structurizr era buena candidata para el modelo C4 como código, pero produce un formato propio que no es Markdown con Mermaid. Mantener dos formatos de diagrama garantizaría que uno de los dos quedara desactualizado.

Continúa en la página siguiente

Tabla 12 — continuación

#	Herramienta	Anillo	Descripción y uso	Justificación
76	Grype	Adoptar	Escaneo de vulnerabilidades conocidas de cada imagen de contenedor en la integración continua. La construcción falla ante cualquier vulnerabilidad de severidad alta o crítica que tenga corrección publicada.	Licencia Apache 2.0 y ejecución como binario de línea de órdenes, sin servicio externo ni cuenta adicional (CP-03). Materializa bajo RD-08 la regla de no desplegar una imagen base con vulnerabilidad conocida. Se ejecuta fijado a una versión y verificado por suma de comprobación.
77	Gitleaks	Adoptar	Detección de secretos en cada Pull Request y en el historial de cada repositorio, ejecutada dentro de GitHub Actions. Un hallazgo bloquea la fusión.	Licencia MIT. Se ejecuta el binario de línea de órdenes y no la acción publicada por su autor, que exige clave de licencia para repositorios de una organización; la detección nativa de la plataforma sobre repositorios privados es de pago. Cumple la política de ausencia de secretos de la Sección 7.7 por construcción, sin depender de que nadie se equivoque nunca.
79	Trivy	Evitar	Escáner de vulnerabilidades de imágenes, alternativa a la entrada 76.	Descartado. En marzo de 2026 su ecosistema sufrió un compromiso de cadena de suministro (CVE-2026-33634) que publicó una versión maliciosa de su binario y reescribió casi todas las etiquetas de su acción de GitHub para robar credenciales de las cadenas de integración continua. Un escáner corre con los secretos de la cadena a su alcance, y bajo RD-08 se prefiere la alternativa sin ese antecedente.

3.6 Cuadrante 4 — Técnicas y Prácticas

Cuadro 13: Catálogo del cuadrante Técnicas y Prácticas (entradas 54 a 73 y 78).

#	Práctica	Anillo	Descripción y uso	Justificación
54	Scrum	Adoptar	Marco de trabajo del equipo: sprints de dos semanas, ceremonias y roles definidos [8].	Marco exigido por el curso y adecuado a un alcance que se refina entrega a entrega.
55	GitFlow	Adoptar	Estrategia de ramificación, normada en la Sección 7.2 [9].	RD-05 lo exige. Es además defendible para este equipo: su estructura de ramas de larga vida y fusiones controladas por Pull Request da a un equipo junior una red de seguridad que el desarrollo sobre rama única no ofrece.
56	Conventional Commits	Adoptar	Formato normalizado del mensaje de commit: tipo, alcance y descripción [14].	Hace legible el historial y permite derivar de él las notas de versión.
57	Versionado semántico	Adoptar	Numeración de las versiones de cada repositorio y de cada contrato de servicio [13].	Con nueve repositorios que versionan por separado (RD-06), la numeración debe comunicar por sí sola si un cambio rompe a quien consume.
58	Arquitectura distribuida por contextos acotados	Adoptar	Servicios de grano grueso, desplegados de forma independiente, sin base de datos compartida y comunicados por contrato [12].	Exigida por RD-01. Ofrece independencia real de despliegue sin el costo de coordinación de una descomposición fina. Registrada en ADR-003 y ADR-014.
59	Contenerización integral	Adoptar	Todo componente ejecutable se distribuye como imagen de contenedor.	RD-03 lo exige. Elimina las diferencias de entorno entre los equipos de los siete integrantes y hace que lo que corre en la máquina de un integrante sea lo mismo que corre en la demostración.

Continúa en la página siguiente

Tabla 13 — continuación

#	Práctica	Anillo	Descripción y uso	Justificación
60	Aplicación de doce factores	Adoptar	Configuración por variable de entorno, paridad entre ambientes y procesos sin estado [15].	Es el principio que hace utilizable una misma imagen en los ambientes de la Sección 7.5. Sin él, RD-03 produciría imágenes con valores de un ambiente cocinados dentro.
61	Modelo C4	Adoptar	Documentación de la arquitectura en niveles de contexto, contenedores y componentes, escrita en Mermaid [10].	Permite comunicar la arquitectura a audiencias técnicas y no técnicas con el mismo conjunto de diagramas. El nivel de contenedores adquiere significado literal bajo RD-03.
62	Vistas 4+1	Adoptar	Organización del documento de arquitectura en vistas lógica, de proceso, de desarrollo, física y de escenarios [11].	Exigida por la estructura del SDD del curso. Complementa a C4: C4 aporta los diagramas, 4+1 aporta la organización del discurso.
63	Registro de Decisión Arquitectónica	Adoptar	Registro estructurado de cada decisión arquitectónica relevante [6].	Deja trazabilidad del porqué de cada decisión. Se produce en la Mesa de Arquitectura y se versiona en Markdown dentro del repositorio de documentación.
64	MoSCoW	Adoptar	Priorización del alcance en <i>Must</i> , <i>Should</i> , <i>Could</i> y <i>Won't</i> .	Con cinco sprints (CP-01), hace explícito qué se corta primero si la capacidad no alcanza. Es el único esquema de prioridad del proyecto: el esfuerzo se expresa en puntos de historia, no como una segunda prioridad.
65	Contratos OpenAPI versionados	Adoptar	Especificación explícita de la interfaz de cada servicio, versionada en su repositorio [17].	Con cinco servicios en dos plataformas y repositorios separados, el contrato es el único punto de acuerdo. Sin él, RD-01 y RD-06 producen desincronización silenciosa entre frentes.

Continúa en la página siguiente

Tabla 13 — continuación

#	Práctica	Anillo	Descripción y uso	Justificación
66	Accesibilidad WCAG 2.2 nivel AA	Adoptar	Criterio de aceptación transversal de toda pantalla: contraste, foco visible, navegación por teclado, tamaño de objetivo táctil y respeto por la preferencia de movimiento reducido [16].	RN-04 incorpora la capacidad de interacción como característica de calidad, y RNF-03 exige que el registro anónimo lo complete cualquier persona. Un sistema público de salud cuya interfaz excluye no cumple su propósito. Se verifica con los instrumentos de la entrada 21, en la revisión de la historia y no al final.
67	Registro de riesgos	Adoptar	Lista viva de riesgos con probabilidad, impacto, responsable y mitigación, revisada en cada cierre de sprint.	El proyecto opera con dos plataformas, nueve repositorios y cuotas gratuitas finitas. Los riesgos identificados en la Sección 4.3 necesitan un lugar donde se les haga seguimiento, no solo donde se enuncien.
68	Pruebas automatizadas orientadas al riesgo	Probar	Estrategia de cobertura: qué se prueba y qué no.	Se priorizan los escenarios de confidencialidad (RNF-01) y de control de acceso territorial (RNF-02), no la cobertura total. Un equipo junior con nueve horas semanales obtiene más valor de diez pruebas sobre lo que no puede fallar que de doscientas sobre lo trivial. Permanece en <i>Probar</i> porque el alcance real se fija en el SDD y se revisa al cierre del Sprint 3.
69	Testcontainers	Evaluar	Pruebas de integración contra una base de datos real levantada en un contenedor efímero.	Disparador: que exista lógica de persistencia suficiente al cierre del Sprint 3. Encaja naturalmente con RD-03 y elimina la divergencia entre la base de prueba y la real.

Continúa en la página siguiente

Tabla 13 — continuación

#	Práctica	Anillo	Descripción y uso	Justificación
70	Pruebas de contrato (Pact)	Evaluar	Verificación automatizada de que el consumidor y el proveedor de una API siguen entendiéndose.	Disparador: que se materialice una desincronización entre dos servicios pese al contrato OpenAPI. Resuelve un problema real entre Java y .NET, pero su curva y su infraestructura de intercambio de contratos son desproporcionadas para un equipo de siete personas en cinco sprints. La alternativa adoptada es la entrada 65.
71	Monolito modular	Evitar	Aplicación única desplegable, con separación interna estricta por módulos.	Descartado por RD-01, que lo prohíbe de forma explícita. Permanece en el catálogo para que la prohibición quede visible y nadie lo reintroduzca como simplificación de última hora.
72	Microservicios de grano fino	Evitar	Descomposición del sistema en muchos servicios pequeños desplegables de forma independiente.	Descartado en ADR-003: el costo de coordinación, despliegue y observabilidad no es asumible bajo CP-01 y CP-02. RD-01 exige distribución, no fragmentación, y lo dice de forma explícita.
73	Desarrollo sobre rama única	Evitar	Integración continua directa sobre una sola rama de larga vida.	Descartado por RD-05, que fija GitFlow. Se documenta aquí porque es la alternativa dominante en la industria y fue considerada: su ventaja es la integración más frecuente; su costo, para este equipo, es que exige una disciplina de pruebas automatizadas que el proyecto todavía no tiene.

Continúa en la página siguiente

Tabla 13 — continuación

#	Práctica	Anillo	Descripción y uso	Justificación
78	Bandeja de salida transaccional expuesta por contrato	Adoptar	El servicio emisor registra cada evento notifiable en su propia base, en la misma transacción que el hecho que lo origina, y el trabajador de Notificaciones lo consume por un contrato interno con arrendamiento de sesenta segundos.	Registrada en ADR-014. Desacopla la entrega del proceso transaccional sin introducir un intermediario de mensajería (la entrada 38 sigue en <i>Evaluar</i>) y sin que el trabajador lea la base de otro servicio, que sería el <i>killer</i> K-04. Impide a la vez donaciones registradas sin aviso y avisos de hechos que no ocurrieron.

3.7 Resumen del catálogo

Cuadro 14: Distribución de las 81 entradas por cuadrante y anillo.

Cuadrante	Adoptar	Probar	Evaluar	Evitar
Lenguajes y Frameworks	23	1	1	1
Datos e Infraestructura	11	0	5	6
Gestión y Colaboración	8	0	1	3
Técnicas y Prácticas	15	1	2	3
Total	57	2	9	13

La proporción es deliberada. Un catálogo con muchas entradas en *Evaluar* sería una lista de deseos; uno sin entradas en *Evitar* no protegería de nada. Las trece entradas descartadas son las que impiden que alguien reintroduzca en el Sprint 4 una decisión que ya se cerró en el Sprint 1.

El anillo *Probar* tiene solo dos entradas, y eso es intencional. *Probar* exige una pregunta abierta y una fecha para responderla. Una tecnología sobre la que no se está evaluando ninguna alternativa no está en prueba: está adoptada, y decirlo de otro modo solo posterga la decisión sin ganar información.

4. Plan de adopción

Un catálogo dice qué se usa; no dice cuándo. Con 57 entradas en anillo *Adoptar*, dos plataformas de ejecución y una capacidad de 126 horas por sprint, el orden de incorporación es tan determinante como la selección. Esta sección convierte el catálogo en un plan.

4.1 Cronograma del proyecto

Cuadro 15: Cronograma de entregas y ubicación de los sprints.

Semana	Sprint	Hito	Entregables
3	—	Lanzamiento	Documento de Proyecto V1
4	—	Herramientas y Planning Sprint 1	Documento de Proyecto V2, Herramientas V1, SRS V1, Backlog V1
6	1	Review Sprint 1 y Planning Sprint 2	SAD V1, DD V1, prototipos de alta fidelidad, Herramientas V2.2, SRS V2, Backlog V2
8	2	Review Sprint 2	SDD V1 con vistas 4+1 y modelo C4, prueba de concepto con su ADR, Infraestructura V1
10	3	Review Sprint 3	Despliegue en QA, informe de pruebas, primer incremento de valor
12	4	Review Sprint 4	Segundo incremento de valor
14	5	Review Sprint 5 y Lanzamiento	Acta de cierre
16	—	Cierre	Versión final

Consecuencia sobre el plan. Los Sprints 1 y 2 son fundacionales: el primer incremento de valor funcional aparece en la Semana 10. Comprometer funcionalidad antes obliga a construir cimientos y producto a la vez, y es la forma más rápida de agotar la capacidad del equipo.

4.2 Adopción por sprint

Cada entrada del catálogo entra en un sprint concreto. Una entrada que no aparece en esta tabla no se incorpora todavía, aunque esté en anillo *Adoptar*: estar adoptada significa que está decidida, no que esté disponible desde el primer día.

Cuadro 16: Plan de incorporación del catálogo, sprint por sprint.

Sprint	Qué se incorpora	Entradas	Criterio de cierre
1 — Fundación del método	Organización y seis repositorios iniciales, GitFlow con protección de rama, backlog en Projects, documentación en Markdown y Mermaid, primeros ADR, prototipo de alta fidelidad de la Aplicación Web.	13–19, 21, 25, 45, 48–50, 54–57, 61–64, 66, 67	Existe un repositorio por componente, con main y develop protegidas, y toda historia del sprint tiene su rama y su Pull Request.

Continúa en la página siguiente

Tabla 16 — continuación

Sprint	Qué se incorpora	Entradas	Criterio de cierre
2 — Esqueto ejecutable	Los tres repositorios que añade ADR-014. Los cinco servicios, el gateway, la Aplicación Web y el proxy de borde arrancan contenedores; cada servicio con base tiene su base, sus roles de propietario y de servicio, su trabajo de migración y su contrato. Cadenas de construcción de las dos plataformas, con imágenes de ejecución mínimas y sondas de salud sin intérprete. Integración continua mínima con detección de secretos y escaneo de imágenes, y publicación de imágenes. Endurecimiento de contenedores y redes por zona. Prueba de concepto del gateway. Verificación de acceso externo a la máquina de QA.	1–3, 6–10, 20, 26–28, 30, 33, 46, 58–60, 65, 74, 76–78, 81	Un solo comando levanta los doce contenedores de Desarrollo en la máquina de cualquier integrante, en diez minutos o menos, después de que los cuatro trabajos de migración terminan con éxito, y todos responden su verificación de salud. Prometheus y Grafana, que completan los catorce contenedores de QA, se verifican en el Sprint 4, cuando entran las entradas 31 y 32. Un Pull Request de ensayo con un secreto y otro con una imagen base vulnerable quedan bloqueados. La máquina de la entrada 34 está aprovisionada y es alcanzable desde fuera de la red institucional.
3 — Primer incremento y seguridad	Gateway operativo, emisión y validación del token, autorización territorial, primeras pruebas automatizadas, despliegue en QA.	4, 5, 11, 22, 23, 29, 34, 47, 68	El escenario de RNF-02 se puede demostrar de extremo a extremo desde fuera de la red institucional: un usuario territorial pide datos ajenos, recibe denegación y el intento queda en la bitácora.
4 — Observabilidad y segundo incremento	Instrumentación de métricas, recolección, tableros y alertas. Notificaciones asíncronas. Primera campaña de medición de carga sobre la pila de medición.	12, 31, 32, 80	RNF-04 se verifica con una medición, no con una estimación, producida por una simulación de Gatling versionada y repetible, y el contador de accesos denegados por jurisdicción alimenta la evidencia de RNF-02.

Continúa en la página siguiente

Tabla 16 — continuación

Sprint	Qué se incorpora	Entradas	Criterio de cierre
5 — Esta- bilización y cierre	No entra tecnología nueva. Se consolida, se corrige y se cierra el estado de las entradas en anillo <i>Probar</i> y <i>Evaluar</i> .	Ninguna	Cada entrada en <i>Pro- bar</i> y <i>Evaluar</i> queda resuelta: adoptada con su ADR, o retirada con la constancia de que su disparador no se acti- vó.

Regla de carga. El Sprint 2 es el más cargado del plan: dos plataformas de ejecución, sus dos cadenas de construcción, la primera cadena de integración continua y la verificación de acceso a la máquina de QA. Por esa razón el gateway y la observabilidad se separan de forma deliberada hacia los Sprints 3 y 4. Concentrar contenerización, integración continua, gateway y métricas en un mismo sprint es el error de planeación más previsible de este proyecto, y se evita por diseño.

4.3 Riesgos de adopción

Los riesgos siguientes nacen de las decisiones de este documento, no del producto. Se administran con la entrada 67 y se revisan en cada cierre de sprint.

Cuadro 17: Riesgos derivados del instrumental y de las políticas.

ID	Riesgo	Impacto si se ma- terializa	Mitigación adoptada
RT-01	Las cuotas gratuitas de minutos de integración continua y de almacenamiento de imágenes no alcanzan para nueve repositorios privados que publican en cada fusión.	La cadena se detiene a mitad de sprint y bloquea toda fusión.	Se publica imagen solo al fusionar en main ; en develop se construye sin publicar. Se conservan las tres últimas etiquetas de cada imagen. Sección 7.5.
RT-02	La máquina virtual institucional no es alcanzable desde fuera de la red de la universidad, o su asignación se demora.	El cliente no puede abrir la demostración y el hito de la Semana 10 no se cumple, aunque el sistema funcione.	Verificación de acceso externo como criterio de cierre del Sprint 2, no del 3. Si no es alcanzable, se escala al cliente por el Product Owner. La máquina personal de un integrante sostiene el ambiente de Desarrollo, pero no el de QA: su disponibilidad depende de que esté encendida.

Continúa en la página siguiente

Tabla 17 — continuación

ID	Riesgo	Impacto si se materializa	Mitigación adoptada
RT-03	La clave privada de firma del token se expone, o su rotación invalida las sesiones activas.	Cualquiera con la clave podría fabricar tokens válidos; una rotación mal ejecutada interrumpe la demostración.	La clave existe solo en el Servicio de Identidad, como secreto de la plataforma y nunca en la imagen. Se rota cada sprint publicando la clave nueva diez minutos antes de firmar con ella y conservando la anterior una hora (ADR-008). La rotación se ensaya en QA con sesiones activas.
RT-04	GitFlow con ramas de larga vida y un equipo junior produce fusiones tardías y conflictos grandes.	Retrabajo al cierre de cada sprint.	Rama por historia, no por sprint; subida diaria al repositorio; Pull Request de máximo 400 líneas. Sección 7.3.
RT-05	Dos plataformas de ejecución duplican el esfuerzo de construcción, prueba, instrumentación y despliegue.	El equipo dedica al instrumental la capacidad prevista para el producto.	Es un costo aceptado, impuesto por RD-02. Se acota repartiendo por contexto acotado y no por capa técnica (ADR-014), de modo que nadie trabaje en las dos plataformas a la vez, y con una plantilla común de servicio .NET compartida por el gateway y los tres servicios de esa plataforma.
RT-06	Las métricas de flujo se calculan a mano y dejan de actualizarse.	El Informe de Evaluación de Sprint pierde su base cuantitativa.	Tabla versionada en redvital-docs con actualización diaria; su vigencia es criterio de cierre de sprint. Sección 8.3.
RT-07	La observabilidad del Sprint 4 compite en capacidad con el segundo incremento de valor.	Se entrega el instrumental y no el producto, o al revés.	La instrumentación de la entrada 12 se deja lista en el Sprint 2 sin recolector; en el Sprint 4 solo se levantan dos contenedores y se importan tableros publicados.
RT-08	La capacidad propuesta supera a la disponible, como ya ocurrió en la planeación del Sprint 1.	Compromiso incumplido ante el cliente.	Priorización MoSCoW obligatoria antes de comprometer, y recorte por la categoría más baja, no por reducción uniforme del alcance de cada historia.

Continúa en la página siguiente

Tabla 17 — continuación

ID	Riesgo	Impacto si se materializa	Mitigación adoptada
RT-09	Una plataforma adoptada sale de su ventana de soporte durante el proyecto sin que nadie lo note.	Se construye sobre una base sin parches, en contradicción con RD-08.	Regla de vigencia de la Sección 3: la ventana de soporte de toda entrada en <i>Adoptar</i> se verifica al cierre de cada sprint junto con el registro de riesgos.
RT-10	Una acción o un binario de terceros de la cadena de integración continua se compromete y exfiltra los secretos de la cadena, como ocurrió con el ecosistema de Trivy en marzo de 2026.	Credenciales del registro de imágenes o claves de despliegue en manos de un tercero.	Toda acción de terceros se fija por identificador de commit y todo binario descargado se verifica por suma de comprobación (Sección 7.5). Los secretos de despliegue no están disponibles en los flujos de Pull Request.
RT-11	Los catorce contenedores de QA superan la memoria de la máquina de la entrada 34, o la puesta en marcha supera los diez minutos de EC-23.	El sistema no arranca en la demostración, o arranca degradado.	Imágenes precompiladas descargadas del registro en lugar de construidas en el arranque, límites de memoria declarados por contenedor, y medición del consumo en reposo y del tiempo de arranque al cierre de cada sprint.

5. Plantillas para la documentación

Todo artefacto del proyecto se produce con una de las plantillas siguientes. El objetivo es que la documentación sea comparable entre sprints y que ningún integrante tenga que decidir el formato cada vez.

Formato de origen. Los artefactos técnicos vivos, es decir ADR, historias, documentación de arquitectura y diagramas, se escriben en Markdown con Mermaid dentro del repositorio de documentación, conforme a RD-04. Los entregables formales al cliente se componen en $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$. Cuando un contenido existe en ambos formatos, la fuente de verdad es la versión en el repositorio.

5.1 Plantilla de Registro de Decisión Arquitectónica

Cuadro 18: Campos obligatorios de un ADR.

Campo	Contenido
Identificador y título	ADR- <i>nnn</i> y frase corta que resume la decisión.

Continúa en la página siguiente

Tabla 18 — continuación

Campo	Contenido
Estado	Propuesta / Aceptada / Rechazada / Reemplazada. Si reemplaza a otro ADR, se indica cuál; si depende de otro, también.
Contexto	Problema o necesidad que obliga a decidir.
Restricciones aplicables	Identificadores RD, RN o CP de la Sección 2 que acotan la decisión.
Opciones consideradas	Alternativas evaluadas. Mínimo dos, con su descarte razonado.
Compromiso evaluado	Qué se gana y qué se sacrifica con la opción elegida. Una decisión sin sacrificio declarado no fue una decisión.
Decisión	Opción seleccionada.
Justificación	Razón concreta para este proyecto, no argumento general de la industria.
Vigencia de lo elegido	Ventana de soporte de la tecnología seleccionada y fecha en que caduca, cuando aplique.
Consecuencias	Implicaciones y riesgos aceptados. Incluye de forma explícita qué entradas del Tech Radar nacen, cambian de anillo o se retiran como resultado.
Responsable y fecha de revisión	Quién implementa y cuándo se revisa la vigencia de la decisión.

5.2 Plantilla de Historia de Usuario

Cuadro 19: Campos obligatorios de una historia del backlog.

Campo	Contenido
Identificador	HU-nnn.
Épica asociada	Épica del backlog a la que pertenece. Las épicas funcionales corresponden uno a uno con los módulos funcionales que define el SRS, que es la fuente de verdad de esa lista; existen además épicas de gestión.
Clase	Épica / Característica / Historia / Historia técnica / Tarea / Investigación / Defecto.
Enunciado	Como [rol], quiero [acción], para [beneficio]. En historias técnicas, el enunciado describe la capacidad habilitada y su beneficio para el proyecto, no para un usuario final.
Criterios de aceptación	Condiciones verificables que determinan que la historia está terminada. Redactadas de forma que otra persona pueda comprobarlas sin preguntar al autor.

Continúa en la página siguiente

Tabla 19 — continuación

Campo	Contenido
Prioridad	MoSCoW: Must, Should, Could o Won't. Es el único esquema de prioridad del proyecto.
Estimación	Puntos de historia. Representan esfuerzo y dificultad; no se usan como una segunda prioridad.
Componente	Servicio o repositorio donde se implementa: Servicio de Identidad, Servicio Institucional, Servicio de Campañas, Servicio de Donación, trabajador de Notificaciones, Gateway, Aplicación Web, Infraestructura o Documentación.
Requerimiento origen	RF o RNF del SRS que la respalda, o restricción RD que la exige en el caso de las historias técnicas. Ninguna historia entra al sprint sin este campo.

Toda historia debe cumplir los criterios INVEST [19] antes de considerarse lista: independiente, negociable, valiosa, estimable, pequeña y verificable. Conforme a RD-07, el trabajo técnico y de gestión también se representa como historia y también se estima: lo que no está en el backlog no consume capacidad reconocida, y por lo tanto no aparece en la velocidad del equipo.

5.3 Plantilla de registro de riesgos

Cuadro 20: Campos del registro de riesgos del proyecto.

Campo	Contenido
Identificador y enunciado	R- <i>nnn</i> y descripción del riesgo en forma de condición, no de temor genérico.
Categoría	Técnico / de gestión / de alcance / externo.
Probabilidad e impacto	Escala de 1 a 5 en cada eje. Su producto ordena el registro.
Disparador	Señal observable que indica que el riesgo se está materializando.
Mitigación	Acción que reduce probabilidad o impacto, con su historia asociada en el backlog.
Plan de contingencia	Qué se hace si ocurre de todas formas.
Responsable y estado	Quién lo vigila y si está abierto, mitigado, materializado o cerrado.

5.4 Estructura del Documento de Diseño de Software

Los documentos técnicos del proyecto siguen la estructura definida en el curso, compatible con [7]:

- Portada, control de versiones, control de cambios, aprobaciones y tablas de contenido.
- Introducción, alcance y objetivos.

- Requerimientos: identificación de usuarios, definición de requerimientos, atributos de calidad conforme a ISO/IEC 25010:2023, restricciones y riesgos.
- Definiciones de arquitectura: modelo de vistas 4+1, niveles, distribución del sistema, tecnologías, patrones de diseño, buenas prácticas y estándares.
- Modelos del sistema: modelo C4 y modelo de datos, con diagrama y diccionario de datos.
- Interfaz de usuario: líneas de diseño gráfico, componentes, principios de usabilidad y accesibilidad, navegación y mockups.

5.5 Otras plantillas vigentes

Cuadro 21: Plantillas complementarias y su uso.

Plantilla	Uso
Acta de Mesa de Arquitectura	Registro de la sesión semanal de decisiones arquitectónicas, incluidos los ADR producidos y el seguimiento a integrantes ausentes.
Informe de Evaluación de Sprint	Reporte de cierre de sprint con indicadores, métricas de flujo, riesgos, plan del sprint siguiente y verificación de vigencia del catálogo.
Marco de indicadores	Definición de los indicadores del proyecto, rúbrica de desempeño de 1 a 5 y métricas ágiles de equipo.
Contrato de servicio (OpenAPI)	Especificación de la interfaz expuesta por un servicio: rutas, esquemas, códigos de error y versión. Se versiona en el repositorio del servicio proveedor.
Ficha de imagen de contenedor	Para cada componente: imagen base con su versión fijada, etiquetas producidas, puertos expuestos, variables de entorno requeridas y punto de verificación de salud.
Plan y reporte de pruebas	Alcance, casos, datos sintéticos empleados y resultado. Incluye de forma obligatoria los casos de RNF-01 y RNF-02.

6. Políticas y lineamientos por fase del proyecto

6.1 Fase 1 — Iniciación y formulación

- Toda solicitud de aclaración al cliente se canaliza exclusivamente a través del Product Owner y se consolida por lotes, no de forma individual.
- Cuando una ambigüedad no pueda resolverse a tiempo, se documenta como supuesto explícito en el documento correspondiente, marcado como pendiente de validación.
- Ningún requerimiento entra a la línea base sin priorización MoSCoW asignada.
- Toda restricción impuesta por el cliente se registra con identificador propio en la Sección 2 y se rastrea hasta la decisión técnica que la satisface, según el Anexo A.
- Las restricciones de diseño se transcriben literalmente en el SRS y en este documento. Reformularlas con palabras propias es la vía más común para perder su alcance sin notarlo.

6.2 Fase 2 — Diseño y arquitectura

- Toda decisión arquitectónica relevante se delibera en la Mesa de Arquitectura y se registra como ADR. No se implementa una decisión no registrada.
- Todo ADR verifica de forma explícita su compatibilidad con las restricciones RD-01 a RD-08 antes de cerrarse, y declara qué entradas del Tech Radar cambian como consecuencia.
- Toda elección de plataforma, marco o motor de base de datos declara su ventana de soporte y la fecha en que caduca. Una tecnología cuyo soporte vence dentro del horizonte del proyecto no se adopta sin plan de migración aprobado.
- Los diagramas C4 se actualizan en el mismo sprint en que se modifica el elemento que documentan, y se escriben en Mermaid dentro del repositorio.
- La frontera entre servicios se define por contexto acotado, no por conveniencia de implementación. Ningún dato pertenece a dos servicios a la vez.
- Todo contrato entre servicios se especifica antes de implementar cualquiera de los dos extremos.
- Toda capacidad transversal, como la notificación o la auditoría, tiene un servicio dueño declarado. En una arquitectura distribuida, un servicio transversal sin dueño es una responsabilidad que nadie implementa.
- La seguridad se considera durante el diseño, no como control posterior. Toda historia que trate datos de donante pasa por revisión de confidencialidad antes de iniciar desarrollo.
- La accesibilidad se define junto con la pantalla, no después. Una pantalla sin orden de foco ni texto alternativo definidos no está diseñada.

6.3 Fase 3 — Construcción

- El código se escribe en inglés; la documentación funcional, en español. Los identificadores del dominio conservan el término del dominio.
- Ninguna credencial, clave o dato sensible se versiona en el repositorio. Se usan variables de entorno y secretos de la plataforma.
- Ningún servicio accede a la base de datos de otro. Toda interacción entre contextos pasa por la API publicada del servicio propietario.
- Todo componente ejecutable se entrega con su archivo de imagen de contenedor. Una funcionalidad que solo corre en la máquina de su autor no está terminada.
- Ninguna dependencia se incorpora sin uso efectivo. Una dependencia declarada y no utilizada se retira antes de la fusión.
- Solo se incorporan dependencias con licencia permisiva o de código abierto compatible con el uso académico del proyecto. Ante una licencia de copia estricta que se propague al producto, se busca alternativa y se deja constancia del descarte.
- Toda incorporación de una herramienta que no esté en el Tech Radar exige, como mínimo, una entrada nueva en este documento aprobada en Mesa de Arquitectura; si tiene consecuencias estructurales, además un ADR.
- El registro de auditoría es obligatorio en toda operación que cambie el estado de una unidad de sangre y en todo intento de acceso denegado.

- Ninguna funcionalidad expone la causa clínica por la cual una unidad fue marcada como no apta. La restricción se implementa en el modelo de datos, no en la interfaz: lo que no se modela no se puede exponer después por descuido.
- Ningún mecanismo de reconocimiento al donante puede tener contraprestación económica, directa ni indirecta, conforme a RN-02. Esto excluye del sistema toda integración con pasarelas de pago, monederos o programas de puntos canjeables.
- Todos los datos de la aplicación son sintéticos y su origen sintético debe ser evidente en la propia pantalla. Se emplean identificadores con prefijo reconocible y nombres de institución inequívocamente ficticios. Nunca se usa el nombre de una institución real, ni siquiera como ejemplo.
- Ninguna interfaz usa emoji como iconografía funcional ni transmite información únicamente por color.

6.4 Fase 4 — Pruebas

- Los escenarios RNF-01, confidencialidad, y RNF-02, control de acceso territorial, tienen caso de prueba obligatorio. Ninguna entrega se considera terminada sin ellos.
- Las pruebas se ejecutan sobre datos sintéticos. No se emplean datos reales de personas bajo ninguna circunstancia.
- Las pruebas de integración se ejecutan contra los servicios en contenedor, no contra procesos locales, para que el resultado sea representativo del entorno desplegado.
- Todo cambio en un contrato entre servicios exige verificar el consumidor antes de fusionar el cambio en el proveedor.
- Toda pantalla nueva se verifica con los instrumentos de la entrada 21 antes de cerrarse: navegación completa por teclado, foco visible y contraste suficiente.
- Todo defecto encontrado se registra en el backlog con su clase correspondiente. Un defecto conocido y no registrado equivale a un defecto oculto.

6.5 Fase 5 — Estabilización y entrega

- El corte interno es el martes. La entrega oficial, el jueves. El miércoles se reserva para ajustes derivados de la revisión interna.
- Ninguna funcionalidad nueva se incorpora después del corte interno.
- Toda entrega se acompaña de la etiqueta de versión de cada repositorio involucrado y de las imágenes correspondientes publicadas en el registro.
- La documentación se actualiza antes de la entrega, no después.
- Se revisa el registro de riesgos, se verifica la vigencia de soporte de las entradas en anillo *Adoptar* y se cierra el estado de las que estaban en *Probar* o *Evaluar* durante el sprint.

7. Políticas y lineamientos de DevOps

7.1 Organización de repositorios

Conforme a RD-06, el proyecto se distribuye en varios repositorios bajo una única organización de GitHub. Cada repositorio versiona y publica de forma independiente, lo que materializa en la práctica la independencia de despliegue que exige RD-01.

Cuadro 22: Repositorios de la organización y su responsabilidad.

Repositorio	Contenido y responsabilidad
redvital-docs	Documentación técnica, ADR, modelos C4 en Mermaid, requerimientos, contexto y registro de riesgos. Única fuente de verdad documental.
redvital-web	Aplicación Web (TypeScript / React).
redvital-gateway	API Gateway (YARP / ASP.NET Core). Enrutamiento, compuerta por ruta y rol, limitación de tasa y entrega de denegaciones al Servicio de Identidad.
redvital-identidad-service	Servicio de Identidad (.NET / ASP.NET Core): usuarios, roles, jurisdicción, sesiones, emisión de tokens y publicación de claves públicas, y su contrato OpenAPI.
redvital-institucional-service	Servicio Institucional (.NET / ASP.NET Core): jerarquía territorial, instituciones, analítica y consulta del auditor, y su contrato OpenAPI.
redvital-campanas-service	Servicio de Campañas (.NET / ASP.NET Core) y su contrato OpenAPI.
redvital-donacion-service	Servicio de Donación (Java / Spring Boot), con su bandeja de salida, y su contrato OpenAPI.
redvital-notificaciones-worker	Trabajador de Notificaciones (Java / Spring Boot). Sin base de datos ni rutas públicas.
redvital-infra	Archivo de orquestación, configuración del proxy de borde, recolección de métricas y tableros.

Reglas transversales. Toda historia que toque más de un repositorio se registra una sola vez en el backlog y se referencia desde cada Pull Request involucrado. Ningún archivo de herramientas de asistencia, configuración personal de un integrante o directorio del entorno de desarrollo se versiona dentro de un repositorio del producto: se declaran en el archivo de exclusiones antes del primer commit.

Nota sobre visibilidad. Los repositorios son privados durante el desarrollo. Es una decisión del equipo, no una restricción del cliente, y tiene un costo declarado: los repositorios privados consumen cuota de minutos de integración continua y de almacenamiento de imágenes, mientras que los públicos no. Se acepta ese costo porque el trabajo es evaluable y otros equipos cursan la misma asignatura. Si RT-01 se materializa, abrir los repositorios es la primera palanca disponible, dado que el sistema opera exclusivamente con datos sintéticos (CP-04).

7.2 Control de versiones y estrategia de ramificación

Se adopta GitFlow en todos los repositorios, conforme a RD-05.

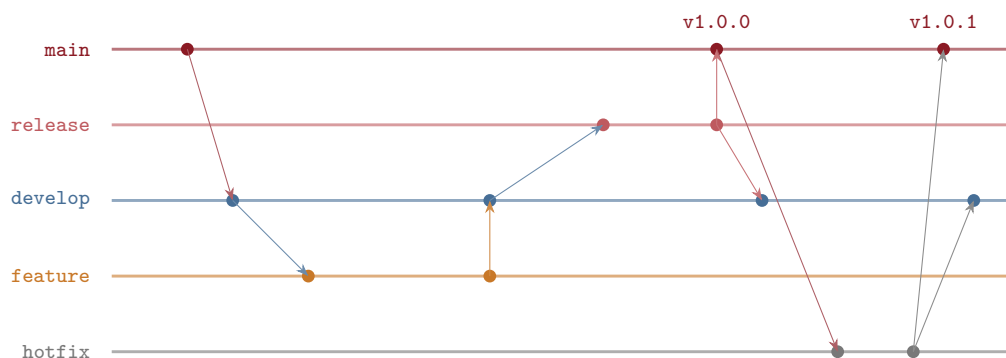


Figura 2: Flujo de ramas adoptado. Las funcionalidades nacen de **develop**, las publicaciones se estabilizan en **release** y se etiquetan en **main**, y las correcciones urgentes retornan a las dos ramas de larga vida.

Cuadro 23: Reglas por rama.

Rama	Lineamiento
main	Refleja lo entregado. Solo recibe fusiones desde release o hotfix . Cada fusión se etiqueta con versión semántica. Protegida: no admite escritura directa ni fusión sin revisión aprobada y verificación en verde.
develop	Rama de integración y base de toda rama de funcionalidad. Debe permanecer en estado construible. Protegida con las mismas condiciones que main .
feature/HU-nmn-descripcion-corta	Una rama por historia, no por sprint ni por integrante. Nace de develop y retorna a develop por Pull Request. Se elimina tras la fusión.
release/x.y.0	Rama de estabilización del sprint. Nace de develop en el corte interno del martes. Solo admite correcciones, nunca funcionalidad nueva. Al cerrar se fusiona a main y de vuelta a develop .
hotfix/x.y.z	Corrección urgente sobre lo entregado. Nace de main y se fusiona a main y a develop . Omitir el retorno a develop es el error clásico de GitFlow: reintroduce el defecto en la siguiente publicación.

Cuadro 24: Convenciones de commit, versionado y frecuencia.

Elemento	Lineamiento
Mensajes de commit	Convención Conventional Commits [14]: tipo(alcance) : seguido de la descripción, por ejemplo feat(donantes) : registro anónimo. Tipos admitidos: feat , fix , docs , refactor , test , chore , build , ci .

Continúa en la página siguiente

Tabla 24 — continuación

Elemento	Lineamiento
Referencia al backlog	Todo commit y todo Pull Request referencia el identificador de la historia. Es el eslabón que hace verificable la cadena de trazabilidad de la Sección 7.8.
Versionado	Semántico [13], por repositorio y de forma independiente. Un servicio no sube de versión porque otro lo haya hecho. Se incrementa la versión mayor cuando cambia el contrato de forma incompatible para quien lo consume.
Frecuencia	Se sube trabajo al repositorio al menos una vez al día. No se acumulan cambios locales durante varios días: es la causa directa de los conflictos grandes que hacen costoso GitFlow.

7.3 Revisión de código

- Todo cambio ingresa a **develop** o a **main** mediante Pull Request. No hay excepciones.
- Cada Pull Request requiere la aprobación de al menos un integrante distinto del autor. El autor no aprueba su propio cambio.
- Un Pull Request no debe superar aproximadamente 400 líneas modificadas. Si las supera, se divide.
- Un Pull Request que modifique un contrato de servicio requiere además la revisión de un integrante del frente responsable del otro extremo.
- La revisión se responde dentro del mismo día hábil para no bloquear al equipo.
- Un Pull Request con la verificación automática en rojo no se fusiona, aunque tenga aprobación humana.
- La revisión comprueba tres cosas de forma explícita antes que el estilo: que no se expone la causa clínica, que el filtrado por jurisdicción está presente donde corresponde, y que no hay secretos en el cambio.

7.4 Contenerización

El sistema tiene doce contenedores de larga vida en Desarrollo y catorce en QA, más cuatro trabajos de migración efímeros y, en QA, los trabajos de respaldo y restauración de cada base, conforme a ADR-014. Los doce de Desarrollo son el proxy de borde, la Aplicación Web, el API Gateway, los cinco servicios —Identidad, Institucional, Campañas, Donación y Notificaciones— y las cuatro bases de datos. QA añade Prometheus y Grafana. Esas cifras condicionan el dimensionamiento de la máquina de la entrada 34 y la medida de EC-23, y por eso se declaran de forma explícita.

Cuadro 25: Lineamientos de contenerización.

Elemento	Lineamiento
Cobertura	Todo componente ejecutable tiene imagen propia. Ninguno se ejecuta como proceso suelto en la máquina de un integrante ni en la máquina de QA.
Construcción	Imágenes multietapa. La etapa de ejecución no contiene compiladores, gestores de paquetes, intérprete de órdenes ni código fuente: <i>chiseled</i> en .NET y <i>distroless</i> con entorno reducido por <i>jlink</i> en Java (entrada 81). Las imágenes oficiales de Caddy, PostgreSQL, Prometheus y Grafana se usan como las publica su proyecto, fijadas por resumen, y la presencia de su intérprete queda registrada como riesgo aceptado en el Documento de Infraestructura V1.
Imagen base	Se fija la versión mayor y menor de la imagen base, y las imágenes de terceros se fijan además por resumen. La imagen de construcción de Eclipse Temurin se fija a la versión de actualización completa (ADR-009). No se usa la etiqueta móvil de última versión, porque destruye la reproducibilidad de la construcción.
Arquitectura de destino	Las imágenes se construyen para la arquitectura de la máquina de QA. Si esa arquitectura difiere de la de los equipos de los integrantes, se produce imagen multiplataforma.
Endurecimiento	Todo contenedor de aplicación corre con usuario no privilegiado, sistema de archivos raíz de solo lectura, capacidades del núcleo retiradas, escalada de privilegios prohibida y límites declarados de CPU y memoria. Son controles baratos y directamente atribuibles a RD-08.
Redes	Una red por zona de confianza, conforme a ADR-014: red_borde , red_aplicacion , una red de datos por base y, en QA, red_observabilidad . Todas, salvo red_borde , se declaran internas y sin salida al exterior. Toda comunicación que no figure en la tabla de aristas permitidas de ADR-014 está prohibida.
Puertos publicados	Solo el proxy de borde publica puertos al anfitrión: 443/tcp, y 80/tcp únicamente para redirigir a HTTPS; dentro del contenedor escucha en 8443 y 8080, de modo que no necesita capacidades del núcleo. Ningún servicio, base ni pieza de observabilidad publica puerto. Cada servicio atiende sus operaciones en el puerto de negocio 8080 y expone salud y métricas en el puerto de gestión 8081, que el gateway no enruta.

Continúa en la página siguiente

Tabla 25 — continuación

Elemento	Lineamiento
Trabajos de la base	Uno de migración por base, que aplica el esquema versionado, carga la porción del conjunto sintético que corresponde a esa base y termina; en QA, además, uno de respaldo diario y uno de restauración. Todos usan el rol propietario de su propia base y no alcanzan ninguna otra; el servicio opera con el rol de servicio, sin definición de esquema.
Etiquetado	Cada imagen se publica con dos etiquetas: la versión semántica y el identificador corto del commit. La segunda permite reconstruir con exactitud qué se desplegó.
Configuración	Toda configuración entra por variable de entorno [15]. Ninguna imagen contiene valores propios de un ambiente.
Secretos	Ningún secreto se escribe en el archivo de imagen ni en el de orquestación. Todo secreto llega al contenedor como archivo montado en <code>/run/secrets</code> : en Desarrollo, generado por el guion de inicialización de <code>redvital-infra</code> fuera del control de versiones; en QA, como secreto de Compose legible solo por el usuario de despliegue. La clave privada de firma del token existe solo en el Servicio de Identidad (ADR-008).
Orquestación	Un único archivo de Compose en <code>redvital-infra</code> levanta el sistema completo en Desarrollo y en QA, con las diferencias expresadas solo como variables y con un perfil de Compose que añade Prometheus y Grafana en QA. Se omite el atributo de versión en la raíz del archivo, obsoleto en la versión vigente de la herramienta.
Verificación de salud y arranque	Todo servicio expone en su puerto de gestión un punto de verificación de salud que comprueba que su base responde; el del trabajador de Notificaciones comprueba que alcanza la bandeja del Servicio de Donación. Como las imágenes propias no tienen intérprete de órdenes, la verificación la ejecuta un programa de sonda incluido en la imagen, declarado en forma de ejecución directa. El arranque se encadena por condición de salud en el orden que fija ADR-014: cada base, su trabajo de migración, Identidad, los tres servicios de negocio, el trabajador, el gateway, la Aplicación Web y el proxy de borde.

7.5 Integración y despliegue continuo

Cuadro 26: Lineamientos de integración y despliegue continuo.

Etapa	Lineamiento
Integración continua	Cada repositorio define su propio flujo. En todo Pull Request se ejecutan compilación, análisis estático (entradas 20 y 21), detección de secretos (entrada 77) y pruebas. En los repositorios que producen imagen se construye la imagen y se escanea (entrada 76). En redvital-infra se valida además el archivo de orquestación y se comprueban de forma estática los invariantes de la topología —puertos publicados, pertenencia a redes y redes internas— sobre la composición resuelta. Un flujo en rojo bloquea la fusión.
Acciones de terceros	Toda acción de GitHub Actions que no publique la propia plataforma se fija por el identificador completo de su commit, nunca por etiqueta, y los binarios que se descargan en la cadena se verifican por suma de comprobación. Una etiqueta puede reescribirse; un identificador de commit, no.
Publicación de imágenes	Se construye en toda fusión, pero solo se publica al fusionar en main . Publicar también desde develop agotaría la cuota gratuita de almacenamiento del registro sin aportar nada: nadie despliega desde develop .
Presupuesto de la cadena	Los minutos de ejecución y el almacenamiento de imágenes del nivel gratuito son finitos en repositorios privados. Se conservan las tres últimas etiquetas de cada imagen y se eliminan las anteriores. El consumo se revisa en cada cierre de sprint (riesgo RT-01).
Ambientes	Tres. Desarrollo , con Compose, en la máquina de cada integrante: doce contenedores, certificado de la autoridad interna del proxy de borde y datos sintéticos cargados por los trabajos de migración. QA , desplegado desde main sobre la máquina virtual institucional de la entrada 34: catorce contenedores, es donde se miden los escenarios de calidad y es el único ambiente que se muestra al cliente. Producción , declarada y no construida: su topología de referencia está en el SAD, para que su ausencia sea una decisión y no un olvido.
Pila de medición	Las pruebas de carga nunca se ejecutan contra la instancia de QA que se muestra al cliente. Se ejecutan sobre una pila de medición: la misma composición de QA, levantada con otro nombre de proyecto sobre la máquina de la entrada 34, con la instancia de QA detenida y fuera de las ventanas de demostración. La carga la genera Gatling (entrada 80) desde el equipo de un integrante, fuera de la máquina de QA. Su configuración declara ese origen exento de la escala anónima del límite de tasa (ADR-007).

Continúa en la página siguiente

Tabla 26 — continuación

Etapa	Lineamiento
Despliegue	En la máquina de QA se hace descargando las imágenes etiquetadas del registro y recreando los servicios con Compose. No se compila en el servidor: lo que corre allí es exactamente la imagen que produjo la cadena.
Base de datos	Los cambios de esquema los aplica el trabajo de migración de cada base antes de que arranque su servicio, mediante migraciones versionadas. No se modifica el esquema manualmente en ningún ambiente.
Reversión	Ante un fallo en QA, se redespiega la etiqueta de imagen anterior conocida como estable antes de investigar la causa. La reversión no requiere reconstruir nada ni deshacer migraciones, porque toda migración se escribe en dos tiempos (entrada 30).

7.6 Observabilidad

- Todo servicio y el gateway exponen un punto de métricas y un punto de verificación de salud en su puerto de gestión 8081, que el gateway no enruta y que solo Prometheus raspa. Un componente que no se puede observar no se considera terminado.
- La recolección se hace con Prometheus y la visualización con Grafana, ambos desplegados como contenedores en QA. En Desarrollo la instrumentación existe, pero no se ejecuta recolector.
- Grafana solo se alcanza a través del proxy de borde, con usuarios nominales y tableros aprovisionados de solo lectura. Prometheus no se expone en ningún caso.
- Se instrumentan como mínimo: latencia y tasa de error por ruta, estado del conjunto de conexiones a la base de datos, conteo de accesos denegados por jurisdicción y conteo de denegaciones del gateway que no pudieron entregarse a la bitácora, `auditoria_denegaciones_no_entregadas_total` (ADR-016).
- El indicador de accesos denegados no es solo operativo: es la evidencia cuantitativa de que RNF-02 funciona, y alimenta la verificación del objetivo específico correspondiente.
- Toda petición lleva el identificador de correlación que genera el gateway, y los servicios lo propagan en cada llamada entre ellos. Los registros de la aplicación se consultan por contenedor y se correlacionan por ese identificador. Su agregación centralizada está en anillo *Evaluar* (entrada 35).
- Ningún tablero ni ninguna métrica puede desagregar información hasta identificar a un donante, ni exponer la causa clínica de una unidad. RN-01 aplica también a la observabilidad, y es el punto donde con más facilidad se filtra sin darse cuenta.
- Los registros de la aplicación y los del proxy de borde no contienen datos identificables de donantes, tokens de acceso ni cookies.

7.7 Seguridad operativa

- El proxy de borde es el único componente alcanzable desde fuera del anfitrión. Termina TLS 1.2 y 1.3, añade las cabeceras de seguridad y rechaza los cuerpos de más de 1 MiB (entrada 74). No valida tokens ni aplica reglas del dominio.

- La autenticación, la compuerta por ruta y rol y la limitación de tasa en sus dos escalas se resuelven en el gateway. El filtrado fino por jurisdicción a nivel de registro se resuelve además en cada servicio: el gateway no puede saber a qué institución pertenece una unidad concreta.
- El token lo emite únicamente el Servicio de Identidad, firmado con RS256, y se valida contra las claves públicas que ese servicio publica (entrada 29). Ante la ausencia de claves públicas, todo componente rechaza las peticiones que exigen sesión. Ningún servicio confía en un encabezado que no haya verificado.
- Los procesos sin usuario —el gateway al entregar denegaciones, el Servicio de Donación en sus procesos programados y el trabajador de Notificaciones— se autentican con credencial de servicio propia, canjeada por un token de servicio de quince minutos (ADR-008).
- Las rutas con prefijo `/internal/` nunca se enrutan por el gateway y solo son alcanzables dentro de la red de aplicación.
- Todo intento de acceso denegado se registra con autor, ruta, jurisdicción solicitada y marca temporal: los que rechaza el gateway, en la serie de auditoría del Servicio de Identidad; los que rechaza un servicio, en su propia serie (ADR-016). Nunca se registra el dato al que se intentó acceder.
- La respuesta a un acceso denegado no confirma la existencia del recurso solicitado ni nombra la jurisdicción ajena.
- Las credenciales se gestionan como secretos montados en `/run/secrets`, nunca en variables de entorno, en el código ni en el historial del repositorio. Si un secreto llega a versionarse, se rota además de eliminarse: borrarlo del historial no lo invalida.
- La clave de firma del token se rota en cada sprint: la clave nueva se publica diez minutos antes de firmar con ella y la anterior se conserva publicada una hora, de modo que ninguna sesión viva se invalida.
- Cada base tiene tres roles: el superusuario, que solo usa el guion de inicialización; el propietario, que solo usan los trabajos de migración, respaldo y restauración; y el de servicio, con que opera el servicio, sin definición de esquema y con solo inserción y lectura sobre las tablas de solo anexo. Ningún servicio usa superusuario.
- Ninguna plataforma fuera de su ventana de soporte de seguridad permanece en anillo *Adoptar*. Es la aplicación operativa de RD-08 y se verifica al cierre de cada sprint.
- El acceso administrativo a la máquina de QA se limita a los integrantes con rol de DevOps, y se hace por clave, no por contraseña.
- Ningún volcado de base de datos, aun con información sintética, se comparte por canales no institucionales.
- Se revisan las alertas de dependencias vulnerables que reporta la plataforma (entrada 47) y los hallazgos del escaneo de imágenes (entrada 76), y su atención se registra como historia en el backlog conforme a RD-07.

7.8 Trazabilidad entre artefactos

Toda unidad de trabajo debe poder rastrearse en ambos sentidos: del requerimiento al artefacto desplegado, y del artefacto desplegado al requerimiento que lo justifica.

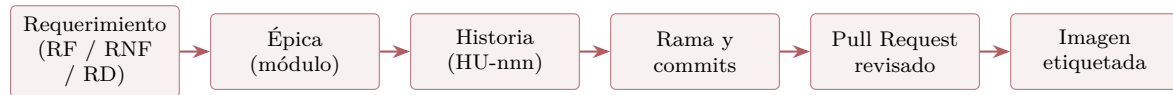


Figura 3: Cadena de trazabilidad entre artefactos del proyecto. La etiqueta de la imagen desplegada permite recorrer la cadena en sentido inverso hasta el requerimiento de origen.

8. Gestión del backlog y métricas

8.1 Herramienta y estructura

El backlog se administra en GitHub Projects, sobre los Issues de los repositorios de la organización. La decisión de concentrar código y backlog en la misma plataforma responde a que la trazabilidad exigida en la Sección 7.8 se vuelve automática: un Issue queda vinculado a su rama, a su Pull Request y a la fusión que lo cierra, sin sincronización manual.

Cuadro 27: Estructura y reglas del backlog.

Elemento	Lineamiento
Épicas	Las épicas funcionales corresponden uno a uno con los módulos funcionales que define el SRS, que es la fuente de verdad de esa lista y de su numeración. Existen además épicas de gestión, que no son módulos funcionales pero sí consumen capacidad y por lo tanto se estiman igual.
Correspondencia con el código	El nombre y el número del módulo deben coincidir entre SRS, épica del backlog y directorio del código. Una discrepancia rompe la cadena de trazabilidad en su eslabón más visible, y se trata como defecto.
Alcance del backlog	Conforme a RD-07, entra al backlog todo el trabajo: funcionalidad, trabajo técnico, infraestructura, documentación y gestión. Ningún trabajo se ejecuta fuera del tablero.
Etiquetas obligatorias	Clase de la historia, componente o repositorio, prioridad MoS-CoW y requerimiento de origen.
Estimación	Puntos de historia, asignados en la sesión de planeación. El trabajo técnico se estima igual que el funcional.
Historia entre repositorios	Se registra una sola vez, en el repositorio del componente que la lidera, y se referencia desde los Pull Request de los demás. Duplicarla rompe el conteo de la velocidad.

8.2 Definición de listo y definición de terminado

Cuadro 28: Definición de listo (DoR) y definición de terminado (DoD).

Criterio	Condiciones
Listo — una historia puede entrar a un sprint cuando	Tiene requerimiento de origen, prioridad MoSCoW, componente asignado y criterios de aceptación verificables por alguien distinto del autor. Está estimada. Cumple INVEST. Si toca datos de donante, pasó la revisión de confidencialidad de la Fase 2. Si depende de un contrato entre servicios, ese contrato ya está especificado. Si pertenece a M1 y otorga, muestra o describe un reconocimiento al donante, pasó la revisión de ausencia de contraprestación económica, directa o indirecta, conforme a RN-02.
Terminado — una historia se cierra cuando	Código fusionado a <code>develop</code> por Pull Request aprobado; verificación automática en verde, incluidos el análisis estático y la verificación de accesibilidad; imagen construida; criterios de aceptación comprobados; y documentación y diagramas afectados actualizados en el mismo sprint.

8.3 Métricas de flujo

Limitación verificada. GitHub Projects ofrece gráficas sobre el estado del tablero, pero no produce por sí solo la velocidad del equipo, la gráfica de trabajo pendiente ni el diagrama de flujo acumulado con la desagregación que exige el Informe de Evaluación de Sprint. Es un costo real de haber descartado Jira, y se asume de forma explícita en lugar de descubrirse a mitad de sprint.

En consecuencia, el Scrum Master mantiene en `redvital-docs` una tabla versionada con el estado diario del sprint, a partir de la cual se calculan los tres indicadores. Que la tabla esté versionada tiene una ventaja sobre el cálculo automático: el historial del sprint queda auditable y no se sobrescribe cuando alguien mueve una tarjeta hacia atrás. Su actualización es criterio de cierre de sprint, porque una métrica que se reconstruye de memoria el último día no mide nada.

Cuadro 29: Indicadores de flujo del proyecto y su fuente.

Indicador	Definición y fuente
Velocidad	Puntos de historia terminados por sprint, según la definición de terminado. Se calcula de la tabla versionada. Solo cuenta lo terminado, nunca lo avanzado.
Trabajo pendiente	Puntos restantes por día del sprint, contra la línea ideal. Detecta el compromiso excedido en la primera semana, no en la última.
Flujo acumulado	Conteo diario de historias por estado. Hace visible dónde se acumula el trabajo: si crece la columna de revisión, el cuello de botella es la revisión de código, no el desarrollo.
Capacidad comprometida	Puntos comprometidos frente a la capacidad estimada de 126 horas por sprint. Su desviación alimenta el riesgo RT-08.

Continúa en la página siguiente

Tabla 29 — continuación

Indicador	Definición y fuente
Equivalencia del punto de historia	Horas de capacidad que representa un punto, calculada al cierre de cada sprint dividiendo las horas efectivamente registradas entre los puntos terminados. Se publica en la tabla versionada junto con los tres indicadores anteriores. Es la conversión que hace comparables la velocidad, expresada en puntos, y la capacidad, expresada en horas; sin ella el indicador de capacidad comprometida no es calculable.

9. Gobierno del documento

Este documento cambia durante el proyecto. Para que siga siendo confiable, su cambio también está normado.

Cuadro 30: Reglas de gobierno de este documento.

Aspecto	Regla
Responsable	La Arquitecta de Software es la responsable del contenido; el Scrum Master, de que la versión vigente esté publicada y de que el equipo la conozca.
Frecuencia de revisión	Al cierre de cada sprint, junto con el registro de riesgos. Una revisión que no cambia nada también se registra: deja constancia de que se miró.
Verificación de vigencia	En cada revisión se comprueba que ninguna entrada en anillo <i>Adoptar</i> haya salido de su ventana de soporte, y que todo disparador del anillo <i>Evaluar</i> siga siendo alcanzable.
Quién puede cambiar un anillo	Solo la Mesa de Arquitectura, y siempre con el ADR o el acta que lo sustente. Nadie cambia un anillo dentro de un Pull Request.
Cómo se incorpora una herramienta nueva	Propuesta en Mesa de Arquitectura, entrada nueva en el catálogo y, si tiene consecuencias estructurales, ADR. Si la herramienta ya está en uso cuando se propone, el hecho se registra como incumplimiento de la política de la Fase 3.
Conflicto con otro documento	Si este documento contradice al contexto maestro o al SRS en una restricción, prevalece el contexto maestro y este documento se corrige. Si contradice a un ADR en una decisión técnica, prevalece el ADR.
Vigencia	El documento caduca con el proyecto. No pretende ser un estándar corporativo permanente, y sus justificaciones están escritas para estas condiciones concretas, no para cualquier proyecto.

A. Matriz de cumplimiento de restricciones

Cada restricción se rastrea hasta las entradas del catálogo y las políticas que la satisfacen. La matriz permite verificar dos cosas a la vez: que ninguna restricción quedó sin instrumento, y que ningún instrumento se adoptó sin restricción que lo justifique.

Se distinguen dos columnas. Los **instrumentos** son las entradas que cumplen la restricción. Las **fronteras** son las entradas en anillo *Evitar* que existen precisamente porque esa restricción las prohíbe: no la satisfacen, la delimitan.

Cuadro 31: Matriz de cumplimiento de las restricciones de diseño y normativas.

ID	Restricción	Instrumentos	Fronteras	Política
RD-01	Solución distribuida, no monolito modular	26, 58, 65, 78	71, 72	Secciones 6 y 7
RD-02	Mínimo un componente Java y uno .NET	1–12, 20, 22	—	Sección 3.3
RD-03	Todo contenerizado	27, 28, 33, 34, 59, 60, 74, 81	39, 42, 43	Sección 7.4
RD-04	Documentación en Markdown con Mermaid	48, 61	53	Sección 5
RD-05	GitFlow	55, 56, 57	73	Sección 7.2
RD-06	Múltiples repositorios en una organización	45, 46, 57, 65	—	Sección 7
RD-07	Todo el trabajo en el backlog	45, 64, 67	52	Sección 8
RD-08	Seguridad como atributo de primer orden	1, 2, 4, 8, 11, 26, 29, 31, 47, 68, 74, 76, 77, 81	40, 41, 79	Secciones 7.4, 7.6 y 7.7
RN-01	Confidencialidad clínica	32, 68	—	Secciones 6 y 7.6
RN-02	Prohibición de remuneración	68	—	Sección 6, Fase 3
RN-03	Procedimientos de la cadena transfusional	26, 30	44	Secciones 6 y 7.5
RN-04	ISO/IEC 25010:2023	21, 62, 66, 67, 80	—	Secciones 5 y 6

Verificación inversa. Las 57 entradas en anillo *Adoptar* se rastrearon una a una hasta una restricción o condición de la Sección 2. No quedó ninguna sin origen. Las entradas 49 y 50, Overleaf y Figma, responden a CP-03 y a la necesidad de producir entregables, no a una restricción del cliente, y así se declara.